

Minimum Weighted Hazard-Exposure Dispatch: ~~Complexity, an Exact Algorithm, A Scheduling~~ ~~View with Machine-Checked Proofs and an~~ ~~FPTAS~~a Wildfire Case Study

Quang-Vinh Dang^{1*}, Minh Ngoc Dinh², Hoang-Viet Vu¹,
Phuc-Son Nguyen³

^{1*}British University Vietnam, Hung Yen, Vietnam.

²Millennia Education, Ho Chi Minh City, Vietnam.

³UEH University, Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): vinh.dq4@buv.edu.vn;
Contributing authors: minh.dinh@maeducation.com;
30066555@st.buv.edu.vn; son.np3393@ueh.edu.vn;

Abstract

We study a dispatch-scheduling problem arising when a single depot must send a vehicle or crew to sites before a spreading hazard (flood, wildfire, contamination plume) reaches them. Each site has a round-trip dispatch time, a hazard-arrival deadline and a criticality weight; the objective is a dispatch order maximizing the total weight of sites served in time. We call this Minimum Weighted Hazard-Exposure Dispatch (MWHED) and show it is exactly the classical problem $\mathbf{1} \parallel \sum \mathbf{w}_j \mathbf{U}_j$, so most of its theory is inherited: weak NP-hardness, a Lawler–Moore dynamic program, a value-scaling FPTAS, and a matroid greedy algorithm for equal dispatch times with any number of vehicles. We present these in a self-contained form for a transportation audience and mark precisely what is classical. What the dispatch setting adds is modest and specific: hardness persists when the hazard-arrival time is the same at every site (with weights proportional to dispatch times); the depot round-trip model is a conservative bound on a chained route; and the FPTAS has a guarantee, best possible for the scheme, that is sharper than the textbook $\mathbf{1} - \epsilon$. Natural heuristics have unbounded worst-case ratio. The algorithms are checked against exhaustive search and the theory is machine-checked in Lean 4. A case study of the 2018 Camp Fire, built from hazard-arrival times that NIST documents for each community, shows in this

model that the exact optimum inherits the assumed response speed: planned at a speed above **68** km/h it is on time in only **16%** of perturbed scenarios, planned at any lower speed it retains **99.7%** of the optimum.

Keywords: scheduling under deadlines, computational complexity, approximation algorithms, matroids, disaster response, formal verification

1 Introduction

A recurring operational problem in infrastructure and emergency-response networks is deciding, from a single depot, in what order to dispatch a vehicle or crew to a set of sites that are each racing against a spreading hazard. A wildfire crew choosing which structures to defend before the fire front arrives, a utility company dispatching repair trucks to substations before a cascading outage propagates to them, and a flood-response team evacuating or servicing neighborhoods before rising water reaches them all face the same underlying decision: given a known rate at which a hazard is spreading, and a known cost of reaching each site from a common depot, in what order should sites be visited to save as much value as possible before the hazard reaches each one in turn?

This decision is a scheduling problem, not merely a routing one. Once every visit is a self-contained round trip from a single depot—the standard topology for a technician, repair-crew, or single-vehicle emergency-response dispatch base—the cost of visiting a site does not depend on which site was visited immediately before it, only on the cumulative time already spent. The question is then purely one of *order*: which sites to reach early, which to defer, and which to concede to the hazard, given that each site i becomes “exposed” once the cumulative dispatch time exceeds its known hazard-arrival deadline d_i , at a loss of w_i .

The three motivating settings above share a further feature that makes the *ordering* decision, rather than the underlying routing or hazard-forecasting problem, the right object of study. In each case, the hazard’s own spread is typically produced by a separate, already well-developed forecasting step—a wildfire-spread simulator driven by fuel load, wind, and terrain; a cascading-failure model for a power network; a hydrological model for a flood plain—so that, by the time a dispatcher is choosing a visiting order, each site’s arrival deadline d_i is already a given input, not something to be modeled jointly with the dispatch decision. Similarly, the round-trip dispatch time p_i is typically fixed by the depot’s location and the local road or access network, and is known before dispatch begins. What remains undetermined, and squarely a combinatorial decision, is the *order* in which a single crew or vehicle should visit sites once p_i and d_i are fixed—exactly the question MWHED isolates. This separation of concerns (forecast the hazard and fix the travel network first; optimize the dispatch order second) mirrors how the wildfire operations-research literature we discuss in Section 2 is typically organized, and is what lets us give a ~~complete~~precise complexity and algorithmic ~~answer to~~picture of the ordering question alone, rather than a heuristic for the joint problem.

We formalize this as Minimum Weighted Hazard-Exposure Dispatch (MWHEd) ~~and give a complete complexity and algorithmic picture. Our contributions are:-~~

~~We show MWHEd is exactly-. It is worth saying at the outset, and Section 3 proves it, that MWHEd is *exactly* the classical single-machine ~~scheduling problem~~ $1||\sum w_j U_j$ of maximizing the weighted number of on-time jobs, ~~and prove it~~-. Most of its theory is therefore inherited from scheduling theory, and we do not claim a new algorithmic technique. The paper is a self-contained and carefully verified treatment of this problem in transportation vocabulary, with a small number of additions that depend on the dispatch setting. We separate the two kinds of content.~~

Classical results, restated with complete proofs (marked “classical” in Table 2):

1. ~~MWHEd is weakly NP-hard by an elementary, self-contained argument: the special case in which every site shares the same hazard deadline is *exactly*-, by a direct reduction from PARTITION that uses only the equal-deadline case, which is the 0/1 knapsack problem (Theorem 3); the hardness of job sequencing with penalties is already in Karp (1972).~~
2. ~~We give an An exact pseudo-polynomial dynamic program (Theorem 5), derived directly derived from an earliest-deadline-first exchange argument that we prove from first principles rather than assert (Theorem 5); this is the Lawler-Moore program (Lawler and Moore 1969).~~
3. ~~We adapt the classical A value-scaling technique to obtain a fully polynomial-time approximation scheme (FPTAS -) for MWHEd FPTAS (Theorem 7), so that a solution within any desired factor $1 - \epsilon$ of optimal is computable in time polynomial in the-; approximation schemes for this problem are classical (Sahni 1976; Gens and Leyner 1981).~~
4. ~~A matroid greedy algorithm for equal dispatch times and any number of sites and $1/\epsilon$ vehicles, identical or with different dispatch times (Theorem 8). This is the scheduling of unit-time jobs on identical or uniform machines, for which $O(n \log n)$ algorithms are known (Lawler 1976; Dessouky et al. 1990); we include a self-contained proof.~~
5. ~~We identify a second special case equal round-trip A classification of the restrictions in which some of the dispatch times, i.e., sites equidistant from the depot—that is solvable *exactly* in $O(n \log n)$ time via a matroid greedy algorithm weights and deadlines are constant (Theorem 8), showing that heterogeneity in dispatch cost *alone* does not cause hardness: it is the combination of heterogeneous dispatch cost and heterogeneous criticality, under a shared deadline structure, that does 12); it is an elementary corollary of items 1 and 4 and of Moore’s algorithm.~~

What the dispatch setting adds. These additions are modest, and we state them as such:

6. *A refined FPTAS analysis.* The textbook guarantee $1 - \epsilon$ is not attained: the scheme of Theorem 7 guarantees $\rho_{n,\epsilon} \geq 1/(1 + \epsilon)$, and this constant is best possible for the scheme as stated, without the optional completion step (Proposition 11); rescaling ϵ recovers the classical bound, so the gain is a constant, not an order.

7. ~~We show the natural fallback for a planner who declines to optimize at all—dispatching sites in increasing-deadline order and simply accepting whichever remain on time—has unbounded worst-case loss relative to W^* . The two readings of a deadline.~~ Under the arrival reading (the crew must arrive before the hazard does), MWHED stays weakly NP-hard even when the hazard-arrival time is the same at every site and weights are proportional to dispatch times (Proposition 9); ~~so Theorems 5 and 7 are not merely convenient but necessary for any 4; to our knowledge not stated before); and the depot round-trip model is a conservative bound on a route that chains the sites (Proposition 1), which says when the round-trip premise is appropriate and when it is not (when travel times form a metric).~~
8. ~~Worst-case examples.~~ Dispatching in deadline order, with or without skipping, and a weighted greedy repair all have unbounded worst-case ~~guarantee at all.~~
~~Together, Theorems 3 and 8 give a sharp structural picture of where the difficulty in this problem lives, Theorems 5 and 7 give planners exact and provably near-exact tools for the general, hard case, ratio (Propositions 9 and 10); the examples are elementary, and Proposition 9 shows those tools are not optional: the obvious do-nothing alternative can be made to fail arbitrarily badly. A numerical illustration in Section 5 confirms the FPTAS tracks the true optimum closely well before its worst-case guarantee requires, while a naive dispatch order (no optimization at all, just visiting sites in deadline order) degrades as instances grow. on random instances the greedy repair is within 1% of the optimum on average.~~
9. ~~Verification.~~ Every algorithm is checked against exhaustive search (Section 5.1), and the statements and proofs of the lemmas, theorems and propositions above, except complexity-class and running-time claims, are machine-checked in Lean 4 (see the Data Availability statement for the precise scope; the algorithms’ implementations are checked by testing, not by Lean).
10. ~~A case study on documented data.~~ A Camp Fire instance built from the hazard-arrival times that NIST documents for each community, compared with a chained-route model, with a sensitivity analysis and a sample-average plan (Section 5.7).

To keep the argument concrete ~~rather than purely symbolic~~, Sections 3 and 4 carry a single four-site running example through ~~every result~~ the results, worked out by hand alongside each proof.

The remainder of this paper is organized as follows. Section 2 positions MWHED against the scheduling, ~~real-time-systems, deadline-routing~~ and disaster-response routing literatures ~~literatures and states precisely what is classical~~. Section 3 defines MWHED ~~formally~~ and establishes the earliest-deadline-first ~~structural~~ lemma every later result depends on. Section 4 proves the ~~four theorems and the proposition results~~ above. Section 5 reports ~~four complementary numerical experiments~~ the numerical study and the case study. Section 6 discusses ~~three natural extensions~~ extensions (several vehicles, a moving depot, and uncertain parameters) and precisely what each breaks in our analysis, and Section 7 concludes. Two appendices give an exhaustive hand-verification of the running example and a ~~fully~~ spelled-out derivation of the FPTAS ~~’s approximation-ratio~~ bound.

2 Related work

Scheduling with deadlines. MWHED, once its transportation framing is stripped away, is the classical single-machine problem of maximizing the weighted number of on-time jobs, equivalently minimizing the weighted number of late jobs, $1 \parallel \sum w_j U_j$ in the standard three-field notation (job j has processing time p_j , due date d_j and weight w_j ; $U_j = 1$ if it is late). It is also a single-machine scheduling-with-rejection (order acceptance) problem (Shabtay et al. 2013). Moore (1968) gives an $O(n \log n)$ algorithm for the *unweighted* case ($1 \parallel \sum U_j$): process jobs in earliest-due-date order, and whenever the running schedule becomes infeasible, permanently discard the *scheduled* job with the largest processing time *among those scheduled so far*. This procedure is commonly called (the Moore–Hodgson algorithm. Lawler and Moore (1969) give a general functional-equation (dynamic programming) technique for resource allocation and sequencing problems that specializes to a pseudo-polynomial algorithm for the weighted case; the classification of $1 \parallel \sum w_j U_j$ itself as (weakly Σ_1^P). Lawler (1976) notes that the rule extends to oppositely ordered (agreeable) processing times and weights, $p_i < p_j \Rightarrow w_i \geq w_j$, a class that contains both the equal-weight and the equal-processing-time cases. For the general weighted case, Lawler and Moore (1969) give a dynamic program over jobs in due-date order and total processing time that runs in $O(nP)$ time, $P = \sum_j p_j$; this is our Theorem 5. The problem is weakly NP-hard is standard in the scheduling-complexity literature, surveyed by Lenstra et al. (1977), and follows directly from the same reduction we give in Section 4: job sequencing with deadlines and penalties is among Karp’s original problems (Karp 1972), and an instance with a common due date is *exactly* a 0/1 knapsack instance, whose NP-hardness is classical (Garey and Johnson 1979). We are not aware of a prior statement of this reduction, or of the (Garey and Johnson 1979); complexity surveys include Lenstra et al. (1977). Approximation schemes are classical too: Sahni (1976) gives rounding-based approximation algorithms for single-processor job sequencing with deadlines, which is the same problem, and Gens and Levner (1981) give a faster scheme; both build on the knapsack schemes of Ibarra and Kim (1975), Lawler (1979) and Kellerer and Pferschy (1999, 2004).

Hence our Theorems 3 and 5, and the scheme of Theorem 7, are re-derivations in our notation, not new techniques. The same is true of Theorem 8: for equal processing times (unit jobs) the weighted number of late jobs is polynomial on one machine, on identical machines and on uniform machines, with $O(n \log n)$ algorithms due to Lawler (1976) and Dessouky et al. (1990); see the surveys Baptiste et al. (2004) and Kravchenko and Werner (2011), and Brucker and Kravchenko (2006) for equal-processing-time *tractable special case* jobs and the weighted number of late jobs. With release dates the single-machine problem with equal processing times is still polynomial (Baptiste 1999), whereas the *unweighted* problem on m identical machines with equal processing times and release dates is NP-hard and $W[2]$ -hard parameterized by m (Heeger and Molter 2025). The classification of Theorem 8, *in transportation-network language*¹² is an elementary corollary of these facts, and the more natural tractable boundary in the literature is Lawler’s agreeable-weights case rather than constancy.

This classical ~~scheduling~~ problem remains an active research topic: ~~Hejl et al. (2022) give a refined pseudo-polynomial dynamic program for the practically important “strongly correlated” regime (where processing times and weights are close in value), and Hermelin et al. (2024) improve the Lawler–Moore-style DP program’s running time via $(\max, +)$ -convolution techniques—an algorithmic refinement orthogonal to , and compatible with, Theorem 5’s DP. Heeger and Hermelin (2024) sharpen the complexity picture further, proving and compatible with it.~~ For the special case $w_j = p_j$, which is the case our hardness reduction uses (minimizing the total processing time of late jobs), there is a fine-grained line of work with near-linear pseudo-polynomial algorithms (Bringmann et al. 2022; Klein et al. 2023; Fischer and Wennmann 2025). Heeger and Hermelin (2024) prove that $1 \parallel \sum w_j U_j$ is $W[1]$ -hard parameterized by ~~either the number of distinct processing times or and also by the number of distinct weights in the instance—a result that, translated into MWHED’s language, says that even a small number ; so, unless $FPT = W[1]$, there is no $f(k) \cdot \text{poly}(n)$ algorithm in the number k of distinct dispatch costs times (or criticality levels does not, by itself, guarantee tractability, sharpening the boundary), and a small number of distinct values does not make the problem easy in the fixed-parameter sense. We make no claim about two or more distinct values beyond this; the case $k = 1$ is the tractable case of Theorem 8 and Remark 6 draw between the fully-heterogeneous and equal-cost cases (or Moore–Hodgson, for equal weights).~~ On the heuristic side, Antonov et al. (2026) give a data-driven (machine-learning-guided) heuristic for the same objective, a more sophisticated relative of the weighted greedy-repair heuristic we use as an empirical baseline in Section 5.

Real-time and deadline-driven systems. Earliest-deadline-first ordering is also ~~the central object of classical central to~~ real-time scheduling theory, where Liu and Layland (1973) show that, for periodic tasks on a single processor, earliest-deadline-first achieves full processor utilization while any fixed-priority policy (such as rate-monotonic scheduling) cannot. That literature’s EDF result and our Lemma 2 are structurally analogous—both establish that ordering by deadline is necessary and sufficient for a natural notion of feasibility—but the objectives differ fundamentally: real-time scheduling asks (Liu and Layland 1973), where the question is whether every task can be met, a feasibility question with a clean necessary-and-sufficient utilization test, whereas MWHED presupposes infeasibility is unavoidable when dispatch costs are large relative to deadlines, and asks instead which weighted subset of sites to sacrifice—the optimization question that drives Theorem 3’s hardness result and has no analogue in the feasibility-testing literature. It is worth noting why the two literatures’ core feasibility results still agree despite this difference in objective: Liu and Layland’s utilization test is a necessary-and-sufficient aggregate condition (a single inequality over all tasks) precisely because their tasks recur periodically forever, so any transient slack evens out over a long enough horizon; MWHED’s sites are each served at most once, so no such aggregate test can substitute for Lemma 2’s site-by-site exchange argument, and no analogous single-inequality feasibility test exists for MWHED’s one-shot dispatch problem—indeed, meets its deadline; MWHED presupposes that some sites will be lost, which is the

optimization question behind Theorem 3 shows that even deciding the size of the best *weighted* feasible subset, let alone a simple aggregate test for it, is NP-hard. More recent real-time scheduling work extends EDF-style analysis to richer task models: Günzel et al. (2022) give a unified schedulability test for self-suspending periodic tasks that recovers EDF (among other policies) as a special case, and Wang et al. (2025) refine the schedulability analysis for that same self-suspending, EDF-like setting. Both retain the periodic, run-forever structure that makes an aggregate utilization test possible; MWHED’s one-shot, single-visit-per-site structure is exactly what rules out an analogous test here.

Approximation. The 0/1 knapsack problem admits a fully polynomial-time approximation scheme via scaling and rounding of item values, due to Ibarra and Kim (1975); ~~we adapt this technique directly to MWHED’s dynamic program in~~, with later improvements (Lawler 1979; Kellerer and Pferschy 1999, 2004; Jin 2019; Chen et al. 2025) ~~; the same technique applies to job sequencing with deadlines (Sahni 1976; Gens and Levner 1981), and Theorem 7. The adaptation is not entirely mechanical: knapsack’s FPTAS scales values and reuses a single time-indexed capacity constraint, whereas MWHED has~~ is this construction written for MWHED. The one point at which the presentation differs from knapsack is that there is no single shared ~~capacity—each site carries its own deadline—so capacity~~ (each site has its own deadline), so the value-indexed ~~dual table of Theorem 7’s proof must track~~ table must test feasibility against each site’s individual d_i , rather than a single global bound, and the case split we make explicit in Appendix B (between the scaling being vacuous or not) has no counterpart in the standard knapsack presentation, where $w_{\max}$ and n play a comparatively simpler role. The classical $O(n^3/\epsilon)$ -style FPTAS we adapt is not the fastest knapsack FPTAS known today: Jin (2019) improve the running time via a more careful scaling scheme, and Chen et al. (2025) give a nearly quadratic-time FPTAS, $\tilde{O}(n + 1/\epsilon^2)$, that is conditionally near-optimal. We adapt the classical (rather than the fastest known) construction because its case structure is what ~~; this is easy in deadline order (Lemma 6). A general framework for when a dynamic program yields an FPTAS is due to Woeginger (2000). Our contribution here is only the sharper analysis of Theorem 7’s proof and Appendix B’s derivation build on directly; porting the faster constructions to MWHED’s per-site-deadline setting (rather than knapsack’s single shared capacity) is a natural avenue for tightening Theorem 7’s runtime bound, with the exactly tight constant of Proposition 11. For several vehicles with a common deadline MWHED contains the multiple knapsack problem, which has a PTAS (Chekuri and Khanna 2005); whether the value-scaling scheme extends to a fixed number of vehicles is open here.~~

Matroids and greedy algorithms. The equal-processing-time ~~special~~ case of Theorem 8 is an instance of the classical problem of scheduling unit-time jobs with deadlines to maximize total profit, whose feasible job sets form a ~~matroid, on which a (transversal) matroid on which the~~ greedy algorithm is ~~provably optimal optimal~~ (Edmonds 1971); we follow the treatment in Korte and Vygen (2018). This matroid structure is exactly what breaks once dispatch costs become heterogeneous, which is

why Theorem 8’s greedy algorithm has no counterpart for general MWHED. Concretely, in the equal-deadline restriction (which Theorem 3 identifies with 0/1 knapsack of capacity D), take three sites with dispatch times 2, 1, 5 and a common deadline $D = 5$: $\{3\}$ (the size-5 site alone) is feasible and already uses the entire capacity, while $\{1, 2\}$ (total size 3) is also feasible; but neither remaining site can be added to $\{3\}$ ($5+2$ and $5+1$ both exceed $D = 5$), so a feasible set of size 1 fails to extend toward a feasible set of size 2 by any element of the ~~latter—precisely~~ latter, which is precisely the matroid exchange property failing. This is why a single large, low-value commitment can block a strictly better pair of smaller commitments, and is the structural reason Theorem 3’s hardness survives even though the matroid case (Theorem 8) is easy. ~~Matroid algorithms themselves remain an active area: Terao (2025) give faster matroid partition algorithms, and Ma et al. (2026) prove a greedy algorithm is simultaneously optimal and fair for a multiagent matroid-upgrading problem, reinforcing that “greedy is optimal on a matroid” continues to be~~

Deadline routing. MWHED sits at the “no-routing” end of the family of selective routing problems with deadlines: orienteering with time windows (Vansteenwegen et al. 2011), the deadline-TSP and its approximation algorithms (Tsitsiklis 1992; Bansal et al. 2004), and the traveling salesman problem with profits (Feillet et al. 2005). In those problems the travel time between consecutive sites depends on the sequence; in MWHED every visit is a round trip from the depot, so the occupancy of a site is order-independent and the problem is a scheduling problem with exact algorithms and guarantees. The price is a ~~productive template well beyond the classical setting~~ Theorem 8 uses modelling premise, which Proposition 1 shows to be conservative when travel times form a metric: a route that chains the sites can only protect more.

~~**Routing and dispatch**~~ **Dispatch under a spreading hazard.** Wildfire suppression is the application area where the “race a spreading hazard” structure has been studied most directly in the operations research literature, and it illustrates the point at which our abstraction departs from that literature’s usual level of detail. Rachaniotis and Pappis (2006) schedule a single fire-fighting resource over several fires using the concept of deteriorating jobs, the model closest in spirit to ours. Harris et al. (2023) model fire spread via a minimum-travel-time principle—the fire’s arrival time at any point is governed by the fastest path a flame front can take through spatially varying fuel and terrain, akin to a shortest-path computation over a continuous landscape—and decompose the resulting joint suppression-and-routing decision via logic-based Benders decomposition, trading exact fire-spread geometry against tractable large-scale optimization. Avci et al. (2024) extend this style of problem to multiple heterogeneous resource types (e.g., ground crews and aircraft with different suppression capabilities and travel speeds), ~~which multiplies the routing decision’s combinatorial complexity without changing its underlying deadline structure.~~ Alvelos et al. (2025) go further, jointly optimizing dispatching, the positioning of resources in anticipation of the fire’s advance, and their routing, in a single integrated model. More recently, Granda et al. (2025) give a mathematical programming formulation for wildfire suppression resource allocation, and, closest in spirit to our own complexity results, Delazeri and Ritt (2026) prove NP-completeness for a

landscape-graph model of wildfire suppression resource allocation, alongside a competitive mixed-integer program and a physics-grounded (Rothermel fire-spread model) instance ~~generator—an generator, an~~ independent hardness result in essentially our application domain ~~;~~ though for a differently structured (graph-based, simultaneous multi-resource) model ~~than MWHEd’s single-vehicle dispatch-ordering question.~~ Two recent systematic reviews position the broader literature: Tezcan and Eren (2025) survey optimization approaches to forest-fire suppression, and Rostamian et al. (2026) review 177 papers on wildfire management optimization under uncertainty and complexity, ~~both confirming that pure complexity-theoretic characterizations of a dispatch-ordering subproblem—as opposed to joint suppression, positioning, and routing models—remain rare in this literature.~~ Across all of these, the fire’s own spatial spread is modeled in detail and coupled tightly to the routing and positioning decisions; MWHEd instead abstracts the hazard’s spread into a per-site deadline ~~(a natural reduction once the hazard’s spread model is fixed and known, as is standard once a fire, flood, or contamination plume—, a reduction that is natural once a simulator has produced arrival-time estimates for each site) and asks the sharper question of what complexity and algorithmic guarantees are available for the resulting pure dispatch-ordering decision. To our knowledge, no prior work in this literature establishes the knapsack-equivalence hardness result, the exact dynamic program, the FPTAS, or the equal-cost tractable case we give here. The abstraction is not innocent when the dispatch itself changes the hazard (as suppression does); the cited joint models exist because of that coupling, and MWHEd applies when the deadlines can be taken as given.~~

Positioning. MWHEd ~~sits at the intersection of these four literatures without duplicating any of them: it inherits its complexity classification from classical scheduling theory ~~;~~ both the founding results and their recent refinements (Moore 1968; Lawler and Moore 1969; Lenstra et al. 1977; Hejl et al. 2022; Hermelin et al. 2024; Heeger and Her~~ and its algorithms from scheduling theory (Moore 1968; Lawler and Moore 1969; Lawler 1976; Dessouky et al. 1990), its ~~algorithmic—approximation~~ toolkit from knapsack approximation ~~and—matroid—theory, including recent speedups (Ibarra and Kim 1975; Korte and Vygen 2018; Jin 2019; Chen et al. 2025; Terao 2025; Ma et al. 2026); its deadline-ordering intuition from real-time systems (Liu and Layland 1973; Günzel et al. 2022; Wang et al. 2025 (Ibarra and Kim 1975; Lawler 1979; Kellerer and Pferschy 1999; Jin 2019; Chen et al. 2025), and its motivating application motivation from disaster-response operations research (Harris et al. 2023; Avcı et al. 2024; Alvelos et al. 2025; Granda et al. 2025; Delazeri and Ritt 2026; Tezcan and Her~~); but the specific problem—and every result we prove about it—is new. Rachaniotis and Pappis 2006; Harris et al. 2023; Avcı et al. 2024; Alvelos et al. 2025; Granda et al. 2025; Delazeri et al. 2026). Its theoretical results are, with the exceptions listed in the introduction, classical ones restated in transportation language; Table 2 in Section 4 summarizes how each of our five main results maps onto this classification. The overall pattern—a transportation reparametrization of a classical scheduling problem, whose hardness, exact algorithm, and approximation scheme all follow by directly adapting standard techniques, while a natural structural restriction (equal dispatch cost) recovers tractability—is, we believe, a template that could apply to other

~~transportation-network problems with a similar deadline-driven structure, not only to the specific hazard-dispatch motivation we use here; Section 6 discusses three such extensions concretely~~ marks which, and the paper’s claim to value rests on the careful, machine-checked treatment, the sharper FPTAS analysis, the analysis of the two deadline readings and of the round-trip premise, and the case study on documented data.

3 Model

Definition 1 (Minimum Weighted Hazard-Exposure Dispatch) An instance of MWHED consists of n sites $1, \dots, n$ and a depot. Each site i has a round-trip dispatch time $p_i \in \mathbb{Z}_{>0}$ (the time to travel from the depot to site i and back, or more generally the time the depot’s single vehicle or crew is occupied while serving site i), a hazard-arrival deadline $d_i \in \mathbb{Z}_{>0}$ (the known time at which a spreading hazard reaches site i), and a criticality weight $w_i \in \mathbb{Z}_{>0}$. A *dispatch order* is a permutation σ of $\{1, \dots, n\}$; the *completion time* of site $\sigma(k)$ is $C_{\sigma(k)}(\sigma) := \sum_{j=1}^k p_{\sigma(j)}$, and site i is *on time* under σ if $C_i(\sigma) \leq d_i$, otherwise *exposed*. The objective is to choose σ maximizing the total on-time weight

$$W(\sigma) := \sum_{i=1}^n w_i \cdot \mathbb{1}[C_i(\sigma) \leq d_i]. \quad (1)$$

Table 1 collects the notation introduced in this definition and used throughout the rest of the paper, for reference.

Table 1 Notation used throughout the paper.

Symbol	Meaning
n	number of sites
p_i	round-trip dispatch time for site i
d_i	hazard-arrival deadline for site i
w_i	criticality weight for site i
σ	a dispatch order (permutation of sites)
$C_i(\sigma)$	completion time of site i under order σ
$W(\sigma)$	total on-time weight of order σ , Eq. (1)
W^*	the optimal (maximum) value of $W(\sigma)$
S	a subset of sites dispatched on time
P	$\sum_{i=1}^n p_i$, the total dispatch time (Theorem 5)
$f(i, t)$	exact DP table: max weight using sites $1..i$, time exactly t
ϵ	FPTAS accuracy parameter, $\epsilon \in (0, 1)$ (Theorem 7)
K	FPTAS weight-scaling factor
w'_i	scaled weight, $\lfloor w_i/K \rfloor$
$g(i, v)$	FPTAS’s value-indexed dual DP table
D_i	latest feasible dispatch position, equal-cost case, $\lfloor d_i/p \rfloor$ slot index, equal-size case, $\min\{n, C(d_i)\}$ (Theorem 8)
m	number of vehicles (MWHED-m, Section 4.4)
ρ_{ϵ}	FPTAS guarantee constant (Theorem 7)

Because every visit returns to the depot before the next dispatch begins, p_i does not depend on which site precedes or follows site i in the ~~order—the order, the~~ defining structural feature that distinguishes MWHED from a routing problem and places it in the scheduling family. This is a modelling premise, not a fact about networks. It is natural for a hub-and-spoke network and for crews that must return to the depot (to re-supply, to hand over, or to be debriefed), and, as Proposition 1 below shows, it is conservative otherwise: a crew that may travel directly between sites can only protect more. Minimizing total *exposure cost* $\sum_i w_i(1 - \mathbb{P}[C_i(\sigma) \leq d_i])$ is equivalent to maximizing $W(\sigma)$, since $\sum_i w_i$ is a constant of the instance; we work with the maximization form throughout.

Assumption 1 (Individually feasible sites) Every site satisfies $p_i \leq d_i$.

This is without loss of generality: a site with $p_i > d_i$ cannot be on time under any dispatch order, even served first, so it can be deleted from the instance in a preprocessing pass without changing the optimal value achievable on the remaining sites. One immediate consequence we use later: the single highest-weight site, dispatched alone, is itself a feasible subset, so $W^* \geq w_{\max} := \max_i w_i$.

Remark 1 (Return and arrival readings of the deadline) Definition 1 counts a site as on time when the vehicle's completion of the whole p_i -long occupancy—the round trip, including the return—happens by d_i (the return reading). When what matters is that the vehicle arrive at the site before the hazard does, only the outbound part $a_i \leq p_i$ of the occupancy must fit: site i is protected iff $C_i - p_i + a_i \leq d_i^{\text{haz}}$, where d_i^{haz} is the hazard-arrival time. This is the same test with deadline $d_i := d_i^{\text{haz}} + (p_i - a_i)$, which for a symmetric trip is $d_i^{\text{haz}} + p_i/2$. Hence the arrival reading is an instance of Definition 1 with shifted deadlines, and every algorithmic result below applies to it unchanged; only the input d_i differs. What changes is the meaning of homogeneity: a constant deadline d_i now corresponds to a hazard-arrival time $d_i^{\text{haz}} = d_i - p_i/2$ that varies with the dispatch time, so Theorem 12 does not by itself say anything about a constant hazard-arrival time; Proposition 4 proves that hardness survives that restriction. The two readings can give different answers on the same data, so a practitioner must state which one the hazard-arrival times refer to; Section 5.7 uses the arrival reading.

Proposition 1 (The depot round-trip model is conservative) Let the depot 0 and the sites lie in a metric space with symmetric distance δ , let the crew travel at speed s , and put $a_i := \delta(0, i)/s$ and $p_i := 2a_i$. Under the arrival reading, site i is protected iff the crew arrives at i by its hazard-arrival time h_i . Fix an order $(i_1, \dots, i_k)$. In the depot round-trip model the crew arrives at i_j at time $A_j := \sum_{l < j} p_{i_l} + a_{i_j}$; if it travels directly from each site to the next it arrives at time $B_j < A_j$. Hence every site that the round-trip model protects with an order is also protected by the chained route with the same order, and the optimal protected weight of the chained-route problem is at least the optimum W^* of the MWHED instance with $d_i = h_i + p_i/2$.

Proof $B_j = a_{i_j} + \sum_{l=1}^{j-1} \delta(i_l, i_{l+1})/s$, and $\delta(i_l, i_{l+1}) \leq \delta(i_l, 0) + \delta(0, i_{l+1})$ by the triangle inequality, so $B_j \leq a_{i_j} + \sum_{l=1}^{j-1} (a_{i_l} + a_{i_{l+1}}) = 2 \sum_{l \leq j} a_{i_l} + a_{i_j} = A_{j^*}$. $\square$

The inequality is an equality when the sites lie on different branches of a star network centred at the depot, and can be strict by a wide margin when sites are clustered, as in the case study of Section 5.7. The proposition says that, when travel times form a metric (in particular when the direct roads between sites are open), a depot-spoke optimum is a *lower bound* on what a crew can protect, which is the safe direction for planning, and it says when the round-trip premise is costly: when the sites are close to each other relative to their distance from the depot.

The following lemma is the structural fact both the dynamic program of Section 4 and the tractability results depend on. It is the transportation-scheduling analogue of the classical fact that earliest-due-date order minimizes maximum lateness on a single machine.

Lemma 2 (Earliest-deadline-first feasibility) *Fix a subset $S \subseteq \{1, \dots, n\}$. There exists a dispatch order under which every site in S is on time if and only if, when the sites of S are dispatched first (before any site outside S) in order of ~~increasing~~ non-decreasing deadline d_i (~~ties broken arbitrarily~~), every site in S is on time.*

Proof Sufficiency is immediate: the stated order is one such dispatch order. ~~For necessity, suppose S is feasible under some order σ (every site of S on time). We transform σ into the stated order by two kinds of exchanges, each of which preserves feasibility of~~ Conversely, let S be

~~First, suppose some site $j \notin S$ precedes some site $i \in S$ in~~ be on time under some order σ : ~~Moving j to immediately after the last element of S (and shifting the intervening sites earlier by p_j) strictly decreases the completion time of every site of S that came after j , and leaves the completion time of every site of S that came before j unchanged; it therefore preserves feasibility of every site in S , and after repeating this for every $j \notin S$ that precedes some element of S , all~~ let τ be any order that serves the sites of S forms a prefix of the schedule.

~~Second, with S now a prefix, suppose two adjacent sites $i, i' \in S$ appear in the order with i immediately before i' , yet $d_i > d_{i'}$ —the later-deadline site is scheduled first. Let t_0 first, by~~ non-decreasing deadline. Fix $i \in S$ and let J be the ~~cumulative dispatch time immediately before this pair is served (unaffected by how the pair is internally ordered, since it depends only on the sites preceding them). Under the current order, i completes at $t_0 + p_i$ and i' completes at $t_0 + p_i + p_{i'}$; since the order is feasible for~~ set of sites of S , in particular ~~$t_0 + p_i + p_{i'} \leq d_{i'}$. Swap the pair so that i' precedes~~ that τ serves up to and including i ; every ~~other site in S still finishes at the same time as before, i' now completes at $t_0 + p_{i'}$, and;~~ then $d_j \leq d_i$ for all $j \in J$, and the completion time of i now completes at $t_0 + p_{i'} + p_i$ —the ~~same total as before, since the pair's combined processing time is unchanged by reordering it. Because $p_i > 0$ under τ is $C_i(\tau) = \sum_{j \in J} p_j$. Under σ the vehicle serves all of J before the last~~ of them, $t_0 + p_{i'} < t_0 + p_i + p_{i'} \leq d_{i'}$, so i' remains on time ; and $t_0 + p_i + p_{i'} \leq d_{i'} < d_i$, so ~~say j^* completes, so $C_{j^*}(\sigma) \geq \sum_{j \in J} p_j$. Hence~~

$$C_i(\tau) \leq C_{j^*}(\sigma) \leq d_{j^*} \leq d_i,$$

because σ is on time for S and $j^* \in J$. So i remains on time as well. Repeating this adjacent-transposition argument (a bubble sort on the prefix) sorts S into increasing d_i order without ever violating a previously-satisfied deadline is on time under τ ; as i was arbitrary, the claim follows. $\square$

Lemma 2 lets us restrict attention, without loss of generality, to dispatch orders that serve some subset S first, in increasing-deadline order, followed by the remaining (necessarily exposed) sites in any order; the optimization problem becomes that of choosing S to maximize $\sum_{i \in S} w_i$ subject to the feasibility condition of Lemma 2.

Example 1 (A running illustration) Throughout this section we illustrate each result on a single small instance: four sites with round-trip dispatch times $p = (2, 3, 1, 4)$, hazard deadlines $d = (4, 6, 2, 9)$, and criticality weights $w = (5, 8, 3, 10)$, all satisfying Assumption 1. Sorting by increasing deadline gives the order 3, 1, 2, 4 (deadlines 2, 4, 6, 9); Lemma 2 says we may restrict to dispatch orders that serve some subset S first, in exactly this relative order.

Dispatching every site in this order—the naive policy of simply visiting sites earliest-deadline-first, with no further optimization—gives completion times 1, 3, 6, 10 for sites 3, 1, 2, 4 respectively; the first three are on time, but site 4, with deadline 9, arrives at cumulative time 10 and is exposed, losing its full weight $w_4 = 10$. This naive order captures only $w_3 + w_1 + w_2 = 3 + 5 + 8 = 16$ of the total weight 26.

The optimal choice of S is instead $\{1, 2, 4\}$, excluding site 3: dispatched in deadline order 1, 2, 4, completion times are 2, 5, 9, each on time, for a total on-time weight of $W^* = 5 + 8 + 10 = 23$ —strictly better than the naive 16, by deliberately sacrificing site 3 (deadline 2, weight only 3) to make room for the far more valuable site 4. No dispatch order can do better, as Example 3’s dynamic program table confirms exhaustively. This gap between naive earliest-deadline dispatch and the true optimum is exactly the phenomenon what Section 5’s numerical study documents at scale measures on random instances.

4 Complexity and algorithms

Table 2 previews how the five the results of this section fit together, before we state and prove each in turn and marks which are classical (stated here in transportation language, with complete proofs) and which are additions of this paper.

4.1 Hardness

We first isolate the one fact about completion times on which the hardness argument rests. Recall that, by the defining property of MWHED, the occupancy p_i of the vehicle at site i does not depend on which site was served before it.

Fact 1 (Feasibility under a common deadline) Suppose $d_i = D$ for every site, and let $S \subseteq \{1, \dots, n\}$. There is a dispatch order under which every site of S is on time if and only if $\sum_{i \in S} p_i \leq D$.

Schematic illustration of Example 1 (running example)
marker size $\propto$ criticality weight w_i ; layout is abstract, not a real road network

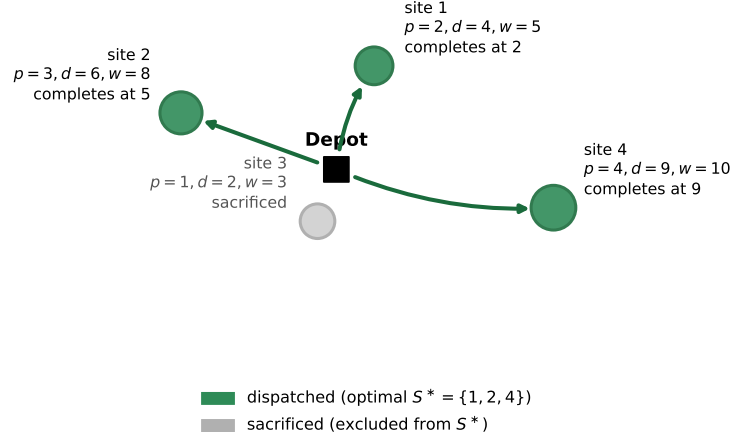


Fig. 1 A schematic illustration of Example 1. Site positions are laid out at a radius proportional to each site's round-trip dispatch time p_i , purely for visual spacing—this is an abstract schematic of the instance, not a real road network or a geographic map (MWHED's theory is instance-agnostic and this paper uses no real-world coordinate data). Marker size is proportional to criticality weight w_i ; green markers and arrows show the optimal dispatch $S^* = \{1, 2, 4\}$ and each site's completion time; the gray marker is site 3, sacrificed to make room for the higher-value site 4.

Proof By Lemma 2 we may serve the sites of S first, in any order (all deadlines are equal, so the earliest-deadline-first order is arbitrary among them). The vehicle is never idle and the occupancies are fixed numbers, so the k -th site served completes at the partial sum of the first k occupancies, and the last site of S completes at the sum $T(S) := \sum_{i \in S} p_i$ of all of their occupancies, whatever the order. This is the only additivity used: it is a statement about the completion time of the last site, not about every site's completion time being additive.

($\Rightarrow$) If every site of S is on time, then in particular the last one is, so $T(S) \leq D$. ($\Leftarrow$) If $T(S) \leq D$, then every site of S completes at a partial sum of positive numbers adding up to $T(S)$, hence at some time $\leq T(S) \leq D$, so every site is on time. $\square$

Theorem 3 (NP-hardness of MWHED) *MWHED is NP-hard. This holds even ~~restricted to instances in which when~~ every site shares a common hazard deadline, $d_i = D - d_i = D$ for all i , and even when $w_i = p_i$ for all i . Precisely, the decision problem “is $W^* > k$?” is NP-complete.*

Proof ~~Consider the restriction $d_i = D$~~ Membership in NP is clear: a dispatch order is a certificate and $W(\sigma)$ is computed in $O(n)$ time. For hardness we reduce from PARTITION (Garey and Johnson 1979): given positive integers $a_1, \dots, a_n$ with $A := \sum_i a_i$, decide whether some $S \subseteq \{1, \dots, n\}$ has $\sum_{i \in S} a_i = A/2$. If A is odd, or if some $a_i > A/2$ (then every subset containing i sums to more than $A/2$ and every other subset to less than $A/2$), the answer is no and we output the fixed one-site instance $p = d = w = 1$ with threshold

$k = 2$, whose optimum is $1 < 2$. So assume that A is even and $a_i \leq A/2$ for all i . By Lemma 2, a subset, and build the MWHED instance with

$$p_i = w_i = a_i, \quad d_i = D := A/2 \quad (i = 1, \dots, n),$$

which satisfies Assumption 1 and is built in polynomial time. By Fact 1 a set S is feasible can be served entirely on time iff $\sum_{i \in S} a_i \leq A/2$, and then its on-time weight is $\sum_{i \in S} w_i = \sum_{i \in S} a_i$. Since only the sites of a feasible set can be on time, $W^* = \max\{\sum_{i \in S} a_i : \sum_{i \in S} a_i \leq A/2\} \leq A/2$, with equality if and only if, dispatched in any order (deadlines being tied, the earliest-deadline-first condition is vacuous), every site in S completes by some subset sums to exactly $A/2$. Thus $W^* > k := A/2$ holds iff the PARTITION instance is a yes-instance. $\square$

Remark 2 (The hard instances are networks) The instances of Theorem 3 arise on a tiny network with a common hazard time. Take a depot D , a hazard origin H and sites $s_1, \dots, s_n$, with edges $D-s_i$ of length a_i and $H-s_i$ of length A , a crew and a hazard that both move at unit speed, and the return reading. As $a_i \leq A/2 < A$, the shortest D -since completion times along a fixed set are additive, this holds exactly when $\sum_{i \in S} p_i \leq D$. The optimization problem

$$\max_{S \subseteq \{1, \dots, n\}} \sum_{i \in S} w_i \quad \text{s.t.} \quad \sum_{i \in S} p_i \leq D$$

is precisely s_i path is the direct edge (any other path costs at least A), so $p_i = 2a_i$, and the hazard reaches every site at time A , so $d_i = A$ for all i . With $w_i = a_i$ this is the instance of Theorem 3 with all lengths doubled. So MWHED is NP-hard on the network $K_{2,n}$ with a hazard front that reaches all sites simultaneously; hardness does not come from an intricate road network.

With general weights the common-deadline restriction is exactly the 0/1 knapsack problem, with p_i the item sizes, w_i the item values, and D the capacity. The (decision version of the) 0/1 knapsack problem is NP-complete (Garey and Johnson 1979), by a polynomial reduction from PARTITION. Hence MWHED restricted to a common deadline is NP-hard, and since this is a special case of the general problem, MWHED itself is NP-hard.

Theorem 3's reduction is deliberately the simplest one available with sizes p_i , values w_i and capacity D , by Fact 1; this is why the equal-deadline case is the natural source of hardness, and why the dynamic program of the next subsection (which is pseudo-polynomial) shows that the hardness is only *weak*. The reduction deliberately uses the simplest possible structure: it isolates *exactly* the source of hardness, namely heterogeneous dispatch costs weighed against heterogeneous criticality under a shared time budget, and reduces the question of what other structure might rescue tractability to what Theorem 8 answers as the source of difficulty, and Theorem 12 shows that this is the only one.

Example 2 (The running example under a common deadline) Suppose the four sites of Example 1 were instead all assigned the same hazard deadline $D = 6$ (same dispatch times and weights). By Theorem 3's reduction Fact 1 this is exactly a 0/1 knapsack instance with

capacity 6, item sizes $p = (2, 3, 1, 4)$, and item values $w = (5, 8, 3, 10)$. Checking every subset whose total dispatch time is at most 6, the best is $\{1, 2, 3\}$, costing $2 + 3 + 1 = 6$ and worth $5 + 8 + 3 = 16$; keeping the single most valuable site instead does worse within the same budget— $\{1, 4\}$ costs 6 but is worth only 15, and $\{3, 4\}$ costs 5 but is worth only 13. Which sites to sacrifice to make room, by weight against size, cannot be read off from either quantity alone; this trade-off is exactly the source of MWHED's hardness.

Theorem 3 uses a common *deadline*. Under the arrival reading of Remark 1 the natural homogeneous case is instead a common *hazard-arrival time*, a hazard that reaches all sites at once, and there the deadlines $d_i = H + p_i/2$ are not constant. Hardness survives this restriction too.

Proposition 4 (Constant hazard-arrival time under the arrival reading) *Let H be an integer and consider the instances with $d_i = H + p_i/2$ for all i (the arrival reading with the same hazard-arrival time H at every site), p_i even integers and $w_i = p_i/2$. MWHED restricted to these instances is weakly NP-hard.*

Proof Feasibility test. Let $S \neq \emptyset$ and serve it by non-decreasing p , equivalently non-decreasing d (Lemma 2). The k -th site i_k completes at $\sum_{j \leq k} p_{i_j}$, so it is on time iff $\mu_k := \sum_{j \leq k} p_{i_j} + p_{i_k}/2 \leq H$, the midpoint of its occupancy. Since $\mu_{k+1} - \mu_k = (p_{i_k} + p_{i_{k+1}})/2 > 0$, only the last site matters, and

$$S \text{ is feasible} \iff \sum_{i \in S} p_i - \frac{1}{2} \max_{i \in S} p_i \leq H. \quad (2)$$

Reduction. Given a PARTITION instance $a_1, \dots, a_n$ with $A = \sum_i a_i$ (if A is odd output a fixed no-instance), build

$$\begin{aligned} p_0 &= 2A + 2, & w_0 &= A + 1; & p_i &= 2a_i, & w_i &= a_i \quad (i = 1, \dots, n); \\ H &= 2A + 1, & d_i &= H + p_i/2. \end{aligned}$$

All p_i are even, $w_i = p_i/2$ for all $i \geq 0$, and $p_i \leq d_i$. We claim that $W^* \geq 3A/2 + 1$ iff the PARTITION instance is a yes-instance. ($\Leftarrow$) If $\sum_{i \in I} a_i = A/2$, take $S = \{0\} \cup I$. Site 0 has the largest p (as $2a_i \leq 2A < 2A + 2$), so by (2) S is feasible iff $(2A + 2) + A - (A + 1) \leq 2A + 1$, which holds with equality, and $w(S) = A + 1 + A/2$. ($\Rightarrow$) Let S be feasible with $w(S) \geq 3A/2 + 1$. If $0 \notin S$ then $w(S) \leq \sum_{i \geq 1} a_i = A < 3A/2 + 1$, so $0 \in S$ and site 0 has the largest p . By (2), $\sum_{i \in S \setminus \{0\}} 2a_i + (2A + 2) - (A + 1) \leq 2A + 1$, i.e. $\sum_{i \in S \setminus \{0\}} a_i \leq A/2$, while $w(S) = A + 1 + \sum_{i \in S \setminus \{0\}} a_i \geq 3A/2 + 1$ forces $\sum_{i \in S \setminus \{0\}} a_i \geq A/2$. Hence equality holds and the a_i with $i \in S \setminus \{0\}$ form a partition. The construction is polynomial, and the problem is only weakly hard by Theorem 5. $\square$

Constant dispatch times (Theorem 8) and constant weights (Moore's algorithm, valid for arbitrary deadlines, hence for $d_i = H + p_i/2$) give polynomial algorithms, so Proposition 4 cannot be strengthened to those restrictions; it shows that, under either reading, a constant hazard-arrival time *alone* never helps.

4.2 An exact pseudo-polynomial algorithm

Theorem 5 (Exact dynamic program) *MWHED can be solved exactly in $O(n \cdot P)$ time and space, where $P := \sum_{i=1}^n p_i$.*

Proof Relabel sites so that $d_1 \leq d_2 \leq \dots \leq d_n$ (Lemma 2 justifies working in this order). Define, for $i = 0, \dots, n$ and $t = 0, \dots, P$,

$f(i, t) :=$ the maximum value of $\sum_{j \in S} w_j$ over $S \subseteq \{1, \dots, i\}$ with $\sum_{j \in S} p_j = t$ and S feasible.

(feasible meaning: dispatching S in increasing-deadline order, every member is on time). Set $f(0, 0) = 0$ and $f(0, t) = -\infty$ for $t > 0$. For $i \geq 1$,

$$f(i, t) = \max \left(f(i-1, t), \begin{cases} [t \leq d_i] \cdot (f(i-1, t-p_i) + w_i) \\ f(i-1, t) \end{cases} \right), \quad (3)$$

where the second term is taken as $-\infty$ if $t < p_i$ or $t > d_i$ with the convention $-\infty + w_i = -\infty$.

We verify (3) by induction on i . The first term corresponds to excluding site i from S . For the second term, suppose $S' \subseteq \{1, \dots, i-1\}$ achieves $f(i-1, t-p_i)$ and is feasible; appending site i after all of S' (site i has the largest deadline among $\{1, \dots, i\}$, so this is consistent with increasing-deadline order) does not change the completion time of any member of S' , so S' 's feasibility is preserved, and site i 's own completion time is exactly $t = (t-p_i) + p_i$, on time precisely when $t \leq d_i$. Conversely, any feasible $S \subseteq \{1, \dots, i\}$ with $i \in S$ and total time t restricts, by the same argument, to a feasible $S \setminus \{i\} \subseteq \{1, \dots, i-1\}$ with total time $t-p_i$ achieving at least $f(i, t) - w_i$ at index $i-1$; hence $f(i-1, t-p_i) \geq f(i, t) - w_i$, giving the reverse inequality and confirming (3) computes exactly $f(i, t)$.

The optimal value is $\max_{t=0}^P f(n, t)$, and the achieving subset is recovered by standard DP backtracking. The table has $O(n \cdot P)$ entries, each computed in $O(1)$ time from (3), giving $O(nP)$ total time and space. $\square$

Theorem 5 is pseudo-polynomial, not polynomial: P can be exponential in the input's bit length. This is consistent with Theorem 3's reduction from knapsack, which is itself, whose hard instances need large numbers (the problem is only weakly NP-hard).

Remark 3 (Relation to Lawler–Moore) This is the Lawler–Moore dynamic program (Lawler and Moore 1969) for $1 \parallel \sum w_j U_j$: jobs in due-date order, state equal to the total processing time of the jobs accepted so far, and the single test $t \leq d_i$ for an accepted job. We do not claim it as new. What the proof above adds is a self-contained derivation from the earliest-deadline-first lemma in the notation of the dispatch problem. Refinements of the $O(nP)$ bound exist (see Section 2); they apply verbatim to MWHED.

Example 3 (The dynamic program on the running instance) Table 3 traces $f(i, t)$ for Example 1, relabeled in increasing-deadline order 3, 1, 2, 4 so that index $i = 1$ is site 3, $i = 2$ is site 1, $i = 3$ is site 2, and $i = 4$ is site 4, matching the order Lemma 2 fixes. Reading the last

row, the largest finite entry is $f(4, 9) = 23$; backtracking (3) from this cell recovers exactly $S^* = \{1, 2, 4\}$ in the original site labels, matching Example 1. A dash marks $t > d_i$ for the site just added at row i : once the cumulative time needed to include that site would already exceed $-\infty$: either the time t cannot be reached by any feasible subset of the first i sites (for instance $f(1, 2)$: site 3 alone costs 1), or including site i at that column would violate its own deadline $t \leq d_i$ (for instance $f(4, 10)$, since $d_4 = 9$); in the latter case the recursion can only carry the previous row's value forward, never improve on it, at that column and every column beyond it in the same row.

Algorithm 1 states Theorem 5's construction explicitly, including the backtracking step used to recover an achieving subset S^* , not merely its value.

Algorithm 1 Exact dynamic program for MWHED (Theorem 5)

Require: sites $1, \dots, n$ with $d_1 \leq d_2 \leq \dots \leq d_n$ (reabeled per Lemma 2), with dispatch times p_i , deadlines d_i , weights w_i
Ensure: optimal on-time weight W^* and an achieving subset S^*

- 1: ~~delete every site with $p_i > d_i$ (Assumption 1); relabel the rest so $d_1 \leq \dots \leq d_n$ (Lemma 2)~~
- 2: $P \leftarrow \sum_{i=1}^n p_i$
- 3: $f[0][0] \leftarrow 0$; $f[0][t] \leftarrow -\infty$ for $t = 1, \dots, P$
- 4: **for** $i = 1, \dots, n$ **do**
- 5: **for** $t = 0, \dots, P$ **do**
- 6: $f[i][t] \leftarrow f[i-1][t]$ ▷ exclude site i
- 7: **if** $t \geq p_i$ **and** $t \leq d_i$ **and** $f[i-1][t-p_i] > -\infty$ **then**
- 8: **if** $f[i-1][t-p_i] + w_i > f[i][t]$ **then**
- 9: $f[i][t] \leftarrow f[i-1][t-p_i] + w_i$
- 10: $choice[i][t] \leftarrow \text{TRUE}$ ▷ site i included to reach this value
- 11: **end if**
- 12: **end if**
- 13: **end for**
- 14: **end for**
- 15: $t^* \leftarrow \arg \max_{t \in \{0, \dots, P\}} f[n][t]$; $W^* \leftarrow f[n][t^*]$
- 16: $S^* \leftarrow \emptyset$; $t \leftarrow t^*$
- 17: **for** $i = n, \dots, 1$ **do** ▷ backtrack
- 18: **if** $choice[i][t]$ **then**
- 19: $S^* \leftarrow S^* \cup \{i\}$; $t \leftarrow t - p_i$
- 20: **end if**
- 21: **end for**
- 22: **return** W^*, S^*

4.3 A fully polynomial-time approximation scheme

For every $\epsilon \in (0, 1)$, a dispatch order σ with $W(\sigma) \geq (1 - \epsilon) \cdot W^*$ (where W^* is the true optimum) can be computed in time polynomial in n and $1/\epsilon$.

We adapt the standard value-scaling technique used for the 0/1 knapsack FPTAS (Ibarra and Kim 1975) to The scheme runs the value-indexed dual of the dynamic program of Theorem 5. Fix $\epsilon \in (0, 1)$ and let $w_{\max} := \max_i w_i$. Define the scaling factor $K := \max(1, \epsilon w_{\max}/n)$ and scaled weights $w'_i := \lfloor w_i/K \rfloor$, so that $w'_i \leq n/\epsilon$ for every i and hence $\sum_i w'_i \leq n^2/\epsilon$. instead of the maximum value for each total time, it tracks the minimum total time for each (scaled) value. The next lemma states the recursion precisely; its dominance step is the one non-routine point.

Run the dynamic program of Theorem 5, but in its value-indexed dual form: for $i = 0, \dots, n$ and $v = 0, \dots, \lfloor n^2/\epsilon \rfloor$,

Lemma 6 (Value-indexed table) *Relabel the sites so that $d_1 \leq \dots \leq d_n$, let $w'_i \in \mathbb{Z}_{\geq 0}$, and for $0 \leq i \leq n$ and $v \geq 0$ let*

$$g(i, v) := \text{the minimum total dispatch time } \sum_{j \in S} p_j \text{ over feasible } S \subseteq \{1, \dots, i\} \text{ with } \sum_{j \in S} w'_j = v,$$

with the same feasibility notion and the same recursion structure as (3), transposed to track minimum time for a target scaled value rather than maximum value for a target time; the recursion is

$$g(i, v) = \min \left(g(i-1, v), \lfloor g(i-1, v - w'_i) + p_i \leq d_i \rfloor \cdot (g(i-1, v - w'_i) + p_i) \right),$$

$\mathcal{F}_i(v)$ be the family of feasible $S \subseteq \{1, \dots, i\}$ with $\sum_{j \in S} w'_j = v$ and $g(i, v) := \min \{ \sum_{j \in S} p_j : S \in \mathcal{F}_i(v) \}$ ($\min \emptyset = +\infty$). Then $g(0, 0) = 0$, $g(0, v) = +\infty$ for $v > 0$, and for $i \geq 1$

$$g(i, v) = \begin{cases} \min \{ g(i-1, v), g(i-1, v - w'_i) + p_i \} & \text{if } v \geq w'_i \text{ and } g(i-1, v - w'_i) + p_i \leq d_i, \\ g(i-1, v) & \text{otherwise.} \end{cases} \quad (4)$$

by the same argument as in Theorem 5's proof, with the roles of the time and value axes exchanged. This table has $O(n \cdot n^2/\epsilon) = O(n^3/\epsilon)$ entries, each computed

Proof The sets of $\mathcal{F}_i(v)$ that do not contain i are exactly $\mathcal{F}_{i-1}(v)$. Let $S = S' \cup \{i\}$ with $i \notin S'$. Serving S in $O(1)$ time, so the whole computation runs in time polynomial in n non-decreasing deadline order puts i last, so by Lemma 2 S is feasible iff S' is feasible and $p(S') + p_i \leq d_i$, where $p(S') = \sum_{j \in S'} p_j$. Thus the sets of $\mathcal{F}_i(v)$ that contain i are exactly $S' \cup \{i\}$ with $S' \in \mathcal{F}_{i-1}(v - w'_i)$ and $p(S') \leq d_i - p_i$. The admissibility condition is an upper bound on $p(S')$ and the objective $p(S') + p_i$ is increasing in $p(S')$ (dominance): if some admissible S' exists then a minimizer of p over $\mathcal{F}_{i-1}(v - w'_i)$ is admissible and optimal, and if that minimizer is not admissible no S' is. Hence the minimum over sets containing i is $g(i-1, v - w'_i) + p_i$ if this is finite and at most d_i (and $v \geq w'_i$), and $+\infty$ otherwise. Taking the minimum with the sets that avoid i gives (4). If $w'_i = 0$ the first line returns $g(i-1, v)$, so zero-scaled sites are never selected. $\square$

Theorem 7 (FPTAS) ~~Let $\hat{S}$ be the feasible subset of largest scaled value v with $g(n, v) \leftarrow \infty$~~ $\epsilon \in (0, 1)$, $w_{\max} = \max_i w_i$, $K = \max\{1, \epsilon w_{\max}/n\}$, $w'_i = \lfloor w_i/K \rfloor$, and let $\hat{S}$ be a feasible set maximizing $\sum_{i \in \hat{S}} w'_i$ (Algorithm 2 returns one). It runs in $O(n^3/\epsilon)$ time, and

- (a) if $\epsilon w_{\max} \leq n$ (so $K = 1$), $W(\hat{S}) = W^*$.
- (b) if $\epsilon w_{\max} > n$ and $n \geq 2$, put $y = n/\epsilon$, $N = \lfloor y \rfloor$ and $\varphi = y - N$. Then

$$W(\hat{S}) \geq \rho_{n,\epsilon} W^*, \quad \rho_{n,\epsilon} := \left(1 + \max\left\{\frac{n-1}{y}, \frac{n-2+\varphi}{N}\right\}\right)^{-1},$$

$$\rho_{n,\epsilon} \geq \frac{N}{N+n-1} > \frac{1}{1+\epsilon} > 1-\epsilon.$$

In particular $W(\hat{S}) > (1-\epsilon)W^*$, the classical guarantee, and $W(\hat{S}) > W^*/(1+\epsilon)$; the constant $\rho_{n,\epsilon}$ cannot be improved (Proposition 11). For $n = 1$ the optimum is trivial.

Proof Running time. In both cases $w'_i \leq w_{\max}/K \leq n/\epsilon$, so $V = \sum_i w'_i \leq n^2/\epsilon$ and the table has $O(n \cdot V) = O(n^3/\epsilon)$ cells, each filled in $O(1)$ by Lemma 6.

Case (a). Then $w'_i = w_i$, so $\hat{S}$ maximizes W over feasible sets, i.e. $W(\hat{S}) = W^*$.

Case (b). Now $K = \epsilon w_{\max}/n > 1$ exactly, so $w_{\max} = Ky$. Let $r_i := w_i - Kw'_i \in [0, K]$ and let h be a site with $w_h = w_{\max}$; then $w'_h = N > 1$ and $r_h = K\varphi$. Put $x = W(\hat{S})$ and $\hat{v} = \sum_{i \in \hat{S}} w'_i$, let S^* be the true optimal subset (value $W^* = \sum_{i \in S^*} w_i$). We consider two cases, according to which argument of $K = \max(1, \epsilon w_{\max}/n)$ attains the maximum.

~~Case $\epsilon w_{\max}/n \leftarrow 1$.~~ an optimal set, $A = S^* \setminus \hat{S}$ and $B = \hat{S} \setminus S^*$. Three facts:

- (F1) $x \geq K\hat{v}$, since $w_i \geq Kw'_i$.
- (F2) $\hat{v} > w'_h = N$, since $\{h\}$ is feasible (Assumption 1) and $\hat{S}$ maximizes the scaled value; in particular $\hat{S} \neq \emptyset$.
- (F3) $W^* - x \leq \sum_{i \in A} r_i$. Indeed $\hat{S}$ maximizes the scaled value, so $\sum_B w'_i \geq \sum_A w'_i$ after cancelling $S^* \cap \hat{S}$, and $W^* - x = \sum_A w_i - \sum_B w_i \leq \sum_A w_i - K \sum_B w'_i \leq \sum_A (w_i - Kw'_i)$.

Since $A \cap \hat{S} = \emptyset$ and $\hat{S} \neq \emptyset$, $|A| \leq n-1$. Write $R := \sum_A r_i$; if $A = \emptyset$ then $W^* \leq x$ and there is nothing to prove, so let $A \neq \emptyset$.

- $h \in \hat{S}$. Then $K = 1$, so $w'_i = \lfloor w_i \rfloor = w_i$ for every i (the w_i are integers by Definition 1): the scaling is vacuous, the dynamic program above is exactly Theorem 5's dynamic program, and it returns $\hat{S} = S^*$, so $W(\hat{S}) = W^* \geq (1-\epsilon)W^*$ trivially (as $\epsilon \in (0, 1)$ and $W^* \geq 0$).
 $x \geq w_h = Ky$ and $R \leq K|A| \leq K(n-1)$, so $W^*/x < 1 + (n-1)/y$.
- ~~Case $\epsilon w_{\max}/n \geq 1$~~ $h \in A$. Then $K = \epsilon w_{\max}/n$ exactly (not merely an upper bound), $R = r_h + \sum_{i \in A \setminus h} r_i \leq K(\varphi + n-2)$ and, by (F1)–(F2), $x \geq KN$, so $W^*/x \leq 1 + (n-2+\varphi)/N$.

- $h \notin A \cup \hat{S}$. Then $A, \hat{S} \subseteq \{1, \dots, n\} \setminus \{h\}$ are disjoint with $\hat{S} \neq \emptyset$, so $Kn = \epsilon w_{\max}$ exactly. Since $w'_i \leq w_i/K$, $|A| \leq n-2$, $R < K(n-2)$, $x \geq KN$, and $W^*/x < 1 + (n-2)/N$, which is dominated by the previous case.

This proves $W(\hat{S}) \geq \rho_{n,\epsilon} W^*$. Since $\varphi < 1$ we have $\sum_{i \in S^*} w'_i \geq \sum_{i \in S^*} (w_i/K - 1) \geq W^*/K - n$, and S^* is itself a feasible subset achieving this scaled value or better, so the returned $\hat{S}$ satisfies $\sum_{i \in \hat{S}} w'_i \geq W^*/K - n$. Converting back, since $w_i \leq K(w'_i + 1)$,

$$\begin{aligned} W(\hat{S}) &= \sum_{i \in \hat{S}} w_i \geq K \sum_{i \in \hat{S}} w'_i \geq K \left(\frac{W^*}{K} - n \right) = W^* - Kn \\ &= W^* - \epsilon w_{\max} \geq W^* - \epsilon W^* = (1 - \epsilon)W^*, \end{aligned}$$

using $Kn = \epsilon w_{\max}$ (this case) and $w_{\max} \leq W^*$ (Assumption 1). Either case gives $W(\hat{S}) \geq (1 - \epsilon)W^*$. $\rho_{n,\epsilon} \geq N/(N + n - 1)$, and $N > y - 1 = n/\epsilon - 1 \geq (n-1)/\epsilon$ (as $1/\epsilon > 1$), i.e. $(n-1)/N < \epsilon$, so $N/(N + n - 1) > 1/(1 + \epsilon) > 1 - \epsilon$. (The classical bound also follows directly from (F3): $W^* - x \leq R < Kn = \epsilon w_{\max} \leq \epsilon W^*$, using $w_{\max} \leq W^*$; Appendix B spells this derivation out.) $\square$

Example 4 (The FPTAS on the running instance) Apply Theorem 7 to Example 1 with a deliberately loose $\epsilon = 0.5$. The scaling factor is $K = \max(1, 0.5 \cdot 10/4) = 1.25$, and the scaled weights are $w' = (\lfloor 5/1.25 \rfloor, \lfloor 8/1.25 \rfloor, \lfloor 3/1.25 \rfloor, \lfloor 10/1.25 \rfloor) = (4, 6, 2, 8)$. Re-running the dynamic program on these scaled weights (tracking minimum dispatch time per scaled value, as in the proof) again selects $S = \{1, 2, 4\}$, recovering the true optimum $W^* = 23$ exactly—comfortably inside the guaranteed bound $(1 - \epsilon)W^* = 11.5$. This mirrors Section 5’s finding at scale: the FPTAS’s worst-case bound is rarely tight in practice, even at a loose ϵ . Here the scaling is active ($K > 1$) yet harmless; Section 5.3 shows that this is typical of random instances, while Proposition 11 gives instances on which the guarantee of Theorem 7 is nearly attained.

Algorithm 2 states the FPTAS’s value-indexed construction explicitly. It has exactly the same shape as Algorithm 1, with the roles of the time axis and the (scaled) value axis exchanged, which is why its table size no longer depends on P .

Remark 4 (Zero-scaled sites and an optional completion step) When $K > 1$ a site with $w_i < K$ has scaled weight $w'_i = 0$ and, because Algorithm 2 minimizes dispatch time for each scaled value, such a site is never selected. The bound of Theorem 7 accounts for the weight so lost (at most K per site), and Proposition 11 shows the loss can approach the fraction $1 - 1/(1 + \epsilon)$ of the optimum. A natural post-processing step, which cannot decrease the returned weight and so preserves Theorem 7, is to re-insert rejected sites in decreasing order of weight while the set stays feasible (checkable in $O(n)$ by Lemma 2); we use it as the “completed” variant in Section 5.3.

Algorithm 2 FPTAS for MWHED (Theorem 7)

Require: sites $1, \dots, n$ with $d_1 \leq \dots \leq d_n$, with dispatch times p_i , deadlines d_i , weights w_i , accuracy $\epsilon \in (0, 1)$

Ensure: a dispatch order with on-time weight $\geq (1 - \epsilon)W^* > \rho_{\epsilon} W^* > (1 - \epsilon)W^*$

```
1: delete every site with  $p_i > d_i$ ; relabel so  $d_1 \leq \dots \leq d_n$ 
2:  $w_{\max} \leftarrow \max_i w_i$ ;  $K \leftarrow \max(1, \epsilon w_{\max}/n)$ 
3:  $w'_i \leftarrow \lfloor w_i/K \rfloor$  for  $i = 1, \dots, n$ ;  $V \leftarrow \sum_{i=1}^n w'_i$ 
4:  $g[0][0] \leftarrow 0$ ;  $g[0][v] \leftarrow +\infty$  for  $v = 1, \dots, V$ 
5: for  $i = 1, \dots, n$  do
6:   for  $v = 0, \dots, V$  do
7:      $g[i][v] \leftarrow g[i-1][v]$   $\triangleright$  exclude site  $i$ 
8:     if  $v \geq w'_i$  and  $g[i-1][v-w'_i] < +\infty$  and  $g[i-1][v-w'_i] + p_i \leq d_i$  then
9:       if  $g[i-1][v-w'_i] + p_i < g[i][v]$  then
10:         $g[i][v] \leftarrow g[i-1][v-w'_i] + p_i$ 
11:         $choice[i][v] \leftarrow \text{TRUE}$ 
12:       end if
13:     end if
14:   end for
15: end for
16:  $v^* \leftarrow \max\{v : g[n][v] < +\infty\}$ 
17:  $\hat{S} \leftarrow \emptyset$ ;  $v \leftarrow v^*$ 
18: for  $i = n, \dots, 1$  do  $\triangleright$  backtrack, as in Algorithm 1
19:   if  $choice[i][v]$  then
20:      $\hat{S} \leftarrow \hat{S} \cup \{i\}$ ;  $v \leftarrow v - w'_i$ 
21:   end if
22: end for
23: return  $\hat{S}$ , dispatched in increasing-deadline order
```

4.4 A second tractable special case: equal-size sites

Theorem 3 shows that heterogeneous dispatch costs, weighed against heterogeneous criticality under a shared deadline, already suffice for hardness. The next result shows that the reverse axis—heterogeneous deadlines under a shared dispatch cost—does not. We state it for $m > 1$ vehicles that need not be identical. In *MWHED- m* each of m vehicles, all starting at the depot at time 0, serves a sequence of round trips; each site is served by at most one vehicle; vehicle v is occupied for $p_v > 0$ at every site it serves (different p_v model vehicles of different speeds over the same distance); and a site is on time if its completion time on its vehicle is at most d_i . Identical vehicles are the case $p_v = p$, and MWHED with equal dispatch times is the case $m = 1$. As the introduction and Section 2 note, this is the scheduling of unit-time jobs on identical or uniform machines; we include a self-contained proof.

For readers who do not work with matroids we recall the notions used. A *matroid* on a finite ground set E is a family $\mathcal{I}$ of *independent* subsets of E with (i) $\emptyset \in \mathcal{I}$; (ii) if $B \in \mathcal{I}$ and $A \subseteq B$ then $A \in \mathcal{I}$; and (iii) the *augmentation property*: if $A, B \in \mathcal{I}$ and $|A| < |B|$, then $A \cup \{x\} \in \mathcal{I}$ for some $x \in B \setminus A$. The *greedy algorithm* considers

the elements of E in order of non-increasing weight and keeps an element if the set kept so far plus that element is still independent. The *transversal matroid* of a bipartite graph has as independent sets the subsets of one side that can be matched into the other side; it is used here with sites on one side and (vehicle, position) *slots* on the other. We prove the matroid property we need directly rather than cite it. A *union-find* (disjoint-set) structure maintains a partition of $\{0, 1, \dots, n\}$ under FIND, which returns the representative of a block, and UNION, which merges two blocks. With union by size (or rank) and path compression, a sequence of n operations costs $O(n\alpha(n))$ time (Korte and Vygen 2018), where α is the inverse Ackermann function, which grows more slowly than any iterated logarithm; path compression alone, with arbitrary linking, only gives $O(\log n)$ amortized time per operation, and for this interval structure linear time is possible on a RAM (Gabow and Tarjan 1985).

Theorem 8 (Equal-size sites are tractable for any number of vehicles) ~~If $p_i = p$ for all~~
~~Vehicle $v \in \{1, \dots, m\}$ needs time $p_v > 0$ for every site, so that the j -th site served by v~~
~~completes at time jp_v ; site i (all sites equidistant from the depot, in round-trip dispatch time~~
~~), MWHED is solvable exactly in $O(n \log n)$ time. has deadline d_i and weight $w_i > 0$ and is~~
~~on time iff it completes by d_i . Put $C(t) := \sum_{v=1}^m \lfloor t/p_v \rfloor$, the number of pairs (v, j) with $j \geq 1$~~
~~and $jp_v \leq t$. Then~~

- (a) ~~a set S can be served entirely on time iff $|\{i \in S : d_i \leq t\}| \leq C(t)$ for every $t \geq 0$;~~
- (b) ~~the feasible sets are the independent sets of a matroid;~~
- (c) ~~the greedy algorithm (non-increasing weight; keep a site iff the kept set stays~~
~~feasible) is optimal, and it can be implemented, with an explicit schedule, in~~
 ~~$O(m + n \log n)$ time.~~

~~For identical vehicles, $p_v \equiv p$, we have $C(t) = m \lfloor t/p \rfloor$ and the running time is $O(n \log n)$ for~~
~~every m (and $m \leq n$ may be assumed).~~

Proof ~~With equal dispatch costs, the k~~
~~Step 1: slots, no gaps. Call (v, j) , $j \geq 1$, a slot with~~
~~time $\tau(v, j) := jp_v$. We claim that S is feasible iff there is an injective map π from S to~~
~~slots with $\tau(\pi(i)) \leq d_i$ for all $i \in S$. Given a schedule, let $\pi(i)$ be the slot site i occupies.~~
~~Conversely, given π , serve on each vehicle its sites in increasing order of the position of $\pi(i)$,~~
~~now at positions $1, 2, \dots$: the r -th dispatched site (whichever it is) completes at time kp ,~~
~~regardless of order. Assigning a site to “dispatch position” site served by v has position r at~~
~~most that of its slot, so it completes no later than $\tau(\pi(i)) \leq d_i$.~~

~~Step 2: the counting criterion (a). ($\Rightarrow$) The sites of S with $d_i \leq t$ need distinct slots of~~
~~time at most t , and there are $C(t)$ such slots. ($\Leftarrow$) List all slots by non-decreasing time,~~
 ~~$\sigma_1, \sigma_2, \dots$ (ties arbitrary), and let $s_1, \dots, s_k$ be the sites of S by non-decreasing deadline.~~
~~The sites $s_1, \dots, s_r$ all have deadline at most d_{s_r} , so $r \leq C(d_{s_r})$: at least r slots have time~~
~~at most d_{s_r} , whence $\tau(\sigma_r) \leq d_{s_r}$. The map $s_r \mapsto \sigma_r$ satisfies Step 1. Since $k \leq n$ it uses only~~
 ~~$\sigma_1, \dots, \sigma_n$: only the n earliest slots matter.~~

~~Step 3: matroid (b). Properties (i) and (ii) are clear from (a). Let A, B be feasible~~
~~with $|A| < |B|$, write $N_X(t) := |\{i \in X : d_i \leq t\}|$ and let $0 = \tau_0 < \tau_1 < \dots < \tau_q$ be 0 and~~
~~the distinct deadlines of $A \cup B$. Let a be the largest index with $N_B(\tau_a) \leq N_A(\tau_a)$;~~
~~since $N_B(\tau_q) = |B| > |A| = N_A(\tau_q)$ we have $a < q$ and $N_B(\tau_j) \geq N_A(\tau_j) + 1$ for all $j > a$.~~
~~Subtracting $N_B(\tau_a) \leq N_A(\tau_a)$ from the case $j = a + 1$ shows that B has more sites~~

with deadline exactly τ_{a+1} than A does, so some $x \in B \setminus A$ has $d_x = \tau_{a+1}$. Adding x to A leaves $N(t)$ unchanged for $t < \tau_{a+1}$ and raises it by one for $t \geq \tau_{a+1}$, where $N_A(t) + 1 \leq N_B(t) \leq C(t)$ because B is feasible. By (a), $A \cup \{x\}$ is feasible.

Step 4: greedy is optimal. Let $G = \{g_1, \dots, g_r\}$ be the set the greedy algorithm returns (weights $w_{g_1} \geq \dots \geq w_{g_r}$, ties broken by index), and let $O = \{o_1, \dots, o_s\}$ be any feasible set with $w_{o_1} \geq \dots \geq w_{o_s}$. G is maximal feasible, so augmentation gives $s \leq r$. We claim $w_{g_k} \geq w_{o_k}$ for all $k \leq s$, which proves $w(G) \geq w(O)$. Suppose $w_{g_k} < w_{o_k}$ for some k ; ~~this is therefore feasible for that site if and only if $kp \leq d_i$, i. e.~~ Apply augmentation to $A = \{g_1, \dots, g_{k-1}\}$ and $B = \{o_1, \dots, o_k\}$: some $o_j \in B \setminus A$ makes $A \cup \{o_j\}$ feasible, and $w_{o_j} \geq w_{o_k} > w_{g_k}$. The greedy algorithm examined o_j before g_k , when its kept set was a subset of A ; by property (ii) that set plus o_j was feasible, so greedy kept o_j , i. e. $o_j \in G$ and, being examined before g_k , $k \leq \lfloor d_i/p \rfloor =: D_i$. ~~The problem becomes: choose an injective assignment of a subset of sites to positions $1, \dots, n$, each site $o_j \in A$ —a contradiction.~~

Step 5: implementation (c). (i) Only the $\min\{m, n\}$ fastest vehicles can contribute to $\sigma_1, \dots, \sigma_n$ (a vehicle appears only if its first slot does); select them in $O(m)$ time by linear-time selection. (ii) Generate $\sigma_1, \dots, \sigma_n$ with a heap holding the next slot time jp_v of each vehicle, recording (v, j) for every popped slot: $O(n \log \min\{m, n\})$. (iii) For each site let $D_i := |\{k \leq n : \tau(\sigma_k) \leq d_i\}| = \min\{n, C(d_i)\}$, by binary search in $O(\log n)$; a site with $D_i = 0$ is discarded. ~~Site i assigned to a position at most D_i , maximizing the total weight of assigned sites. This is the classical problem of scheduling can use exactly the slots $\sigma_1, \dots, \sigma_{D_i}$, so by Steps 1–2 a set is feasible iff its sites can be given distinct indices $k_i \leq D_i$, i. e. iff it is feasible for unit-time jobs with deadlines to maximize total profit. The family of subsets that admit such a feasible assignment forms a matroid (the transversal matroid of the bipartite feasibility relation between sites and positions), and on any matroid, the greedy algorithm—here: sort jobs with deadlines D_i on one machine. (iv) Sort the sites by decreasing w_i , and assign each in turn to the latest available position $k \leq D_i$, or leave it unassigned if none is available—returns a maximum-weight independent set, i. e., an optimal solution (Korte and Vygen 2018, Ch. 13). Sorting takes $O(n \log n)$ time, and maintaining the set of available positions under repeated “find and remove the latest available position $\leq D_i$ ” queries can be implemented with a weight and run Algorithm 3: maintain a union–find structure in $O(n\alpha(n))$ additional time, giving $O(n \log n)$ overall. structure over $\{0, \dots, n\}$ in which $\text{FIND}(q)$ is the largest free position $\leq q$ (or 0 if none); to process site i let $q := \text{FIND}(D_i)$; if $q \geq 1$ keep i , assign it the slot σ_q and merge q into $q - 1$, otherwise discard i . To keep the $O(n\alpha(n))$ bound we link by size and store at each tree root r a label $\ell(r)$, the smallest position of its block, so that $\text{FIND}(q)$ returns $\ell(\text{root}(q))$. The rule keeps i exactly when the kept set plus i is feasible. If $q \geq 1$ the construction exhibits a feasible assignment. If $q = 0$, let q^* be the largest $q \geq D_i$ such that positions $1, \dots, q$ are all taken. Every kept site occupying a position $\leq q^*$ has $D \leq q^*$: a kept site with $D > q^*$ would have been given the latest free position $\leq D$, and position $q^* + 1$ was free then (it is free now; if $q^* = n$ no kept site has $D > q^*$ since $D_i \leq n$), so it would sit above q^* . Hence the kept set has q^* sites with $D \leq q^*$ and, as $D_i \leq q^*$, adding i violates the criterion of Step 2. (v) Serve on each vehicle its assigned sites in increasing slot position (Step 1). The cost is $O(m)$ for (i), $O(n \log n)$ for (ii)–(iv) including the sort, and $O(n\alpha(n))$ for the union–find operations: $O(m + n \log n)$ in total. (The $O(n \log n)$ bound also holds with path compression alone.) For identical vehicles the slots are $\sigma_k = (k \bmod m, \lceil k/m \rceil)$, no heap is needed, and the total is $O(n \log n)$. $\square$~~

Remark 5 The proof uses only that C is a non-decreasing integer-valued function and that the slots can be listed in order of time; vehicles that become available at times a_v (slot times

$a_v + jp_v$) are covered as well. Site release dates are *not* covered (Section 6.1), and sites of unequal sizes are NP-hard already for $m = 1$ (Theorem 3).

Example 5 (The matroid greedy on an equal-cost instance) Consider four sites all at the same round-trip distance $p=2$ from the depot, so that $p=2$ for identical vehicles, with deadlines $d = (3, 3, 5, 9)$ and weights $w = (9, 4, 7, 6)$. With one vehicle ($m = 1$) the earliest slots have times 2, 4, 6, 8, so the latest feasible dispatch position for each site is $D_i = \lfloor d_i/p \rfloor = (1, 1, 2, 4)$. Sorted by decreasing weight, the greedy order is site 1 ($w = 9$), site 3 ($w = 7$), site 4 ($w = 6$), site 2 ($w = 4$). Site 1 takes position 1 (its only option); site 3 takes position 2; site 4 takes position 4; site 2 needs a position ≤ 1 , but position 1 is already taken, so site 2 is left unassigned. The greedy solution dispatches sites 1, 3, 4 for a total weight of $9 + 7 + 6 = 22$, sacrificing only the lowest weight site whose deadline conflicts with a higher weight site's — exhaustive; exhaustive search over all 16 subsets confirms that 22 is indeed optimal. With two identical vehicles ($m = 2$) the earliest slots have times 2, 2, 4, 4 and $D = (2, 2, 4, 4)$: site 1 takes position 2, site 3 position 4, site 4 position 3 and site 2 position 1, so all four sites are served, for 26 (vehicle A serves sites 2 and 4, completing at 2 and 4; vehicle B serves sites 1 and 3, completing at 2 and 4).

Algorithm 3 Matroid greedy for equal-size MWHED- m (Theorem 8)

Require: sites $1, \dots, n$, common dispatch time p , with deadlines d_i and weights w_i ; vehicle dispatch times $p_1, \dots, p_m$ (all equal for identical vehicles)

Ensure: an optimal on-time subset S^* and an explicit schedule

- 1: $D_i \leftarrow \lfloor d_i/p \rfloor$ for $i = 1, \dots, n$; generate the n earliest slots $\sigma_1, \dots, \sigma_n$ with times $\tau_1 \leq \dots \leq \tau_n$, using a heap over the vehicles' next slot times
- 2: sort sites into order $\pi_1, \dots, \pi_n$ with $w_{\pi_1} \geq w_{\pi_2} \geq \dots \geq w_{\pi_n}$; $D_i \leftarrow |\{k : \tau_k \leq d_i\}|$ (binary search); discard sites with $D_i = 0$
- 3: $available \leftarrow \{1, \dots, n\}$; **FIND** over $\{0, \dots, n\}$: every q its own block; $S^* \leftarrow \emptyset$; $i \leftarrow \pi_n$; $S^* \leftarrow \emptyset$
- 4: **for** each site i in order of non-increasing w_i **do**
- 5: $q^* \leftarrow \max\{q \in available : q \leq D_i\}$; $q \leftarrow \text{FIND}(D_i) \triangleright$ largest free position $\leq D_i$, or 0
- 6: **if** $q \geq 1$ **then**
- 7: $S^* \leftarrow S^* \cup \{i\}$; $S^* \leftarrow S^* \cup \{i\}$; $available \leftarrow available \setminus \{q^*\}$; assign i to slot σ_q ; $\text{UNION}(q, q-1) \triangleright q$ is now represented by $q-1$
- 8: **end if**
- 9: **end for**
- 10: **return** S^* , dispatched in increasing deadline order each vehicle serving its assigned sites in order of slot time

Remark 6 Theorems 3 and 8 together show that neither heterogeneous dispatch cost nor heterogeneous deadlines alone is responsible for MWHED's hardness: it is specifically the interaction between heterogeneous dispatch cost and heterogeneous criticality, competing

~~under a shared deadline structure, that produces a knapsack-hard problem; Theorem 12 below makes this exact.~~ A dispatch network in which every site is equally far from the depot—a plausible approximation for, e.g., a ring of substations around a central facility, or short-radius urban service territories—remains exactly solvable regardless of how heterogeneous the sites’ hazard deadlines or criticalities are, ~~and regardless of the number of vehicles, identical or not.~~

4.5 The cost of not reconsidering

Theorems 5 and 7 give planners exact and near-exact tools, but a planner ~~who instead falls back on the simplest conceivable heuristic—dispatch might instead fall back on a simple rule. The simplest is to dispatch~~ sites in increasing-deadline order and simply accept whichever happen to still be on time, never skipping a site to make room for a later, more valuable ~~one—receives no guarantee at all~~ (*naive EDD*; a site that is already late still consumes its dispatch time). A slightly smarter rule skips a site that would arrive late (*EDD with skipping*). Neither has any guarantee.

Proposition 9 (Deadline-order dispatch has unbounded worst-case ratio) *For every $r \in (0, 1)$ there is an MWHED instance, satisfying Assumption 1, on which ~~the naive policy’s~~ both *naive EDD* and *EDD with skipping* obtain on-time weight ~~is~~ strictly less than $r \cdot W^*$.*

Proof Fix $r \in (0, 1)$ and choose an integer $W > 1/r$. Consider the two-site instance with $p = (1, 2)$, $d = (1, 2)$, $w = (1, W)$; both sites satisfy Assumption 1 ($p_i = d_i$). Since $d_1 < d_2$, ~~the naive policy dispatches both rules dispatch~~ site 1 first, completing at time $1 \leq d_1$ (on time, weight 1), ~~then site . Site 2 , completing would then complete~~ at time $1 + 2 = 3 > d_2 = 2$ (~~exposed, losing weight W~~); ~~its : naive EDD serves it anyway and it is exposed, and EDD with skipping discards it; either way the~~ total is 1. Dispatching site 2 alone ~~instead~~ completes it at time $2 \leq d_2$, ~~on time, for weight W ; this is feasible and, since $W > 1 \geq$ the value of any subset not containing site 2, it is $W > 1$ this is~~ optimal, so $W^* = W$. The naive policy’s ratio is therefore $W^* = W$. The ratio is $1/W < r$. □

Proposition 9 is not merely a boundary case: the two sites here differ only by a factor of two in dispatch time, yet the naive policy’s loss can be made arbitrarily large by scaling W alone. No fixed constant-factor guarantee is possible for a policy that never reconsiders an early, ~~The loss comes from never giving up an early, low-value commitment against a later, commitment for a later, high-value one—exactly the reconsideration that Theorem 5’s dynamic program and Theorem 7’s FPTAS perform, one; skipping doomed sites removes only the waste of serving a site that is already late.~~ It is natural to ask whether a local repair rule avoids the problem. The weighted generalization of Moore–Hodgson’s rule used as a heuristic in Section 5—process sites in deadline order and, whenever the running schedule becomes infeasible, discard the scheduled site of smallest weight-to-time ratio—does very well on random instances, but it too has no guarantee.

Its output is always feasible, by the same inductive argument as for the unweighted algorithm: removing a site can only shorten the completion time of every site that remains.

Algorithm 4 Weighted greedy repair (heuristic; no guarantee)

Require: sites with $d_1 \leq \dots \leq d_n$ (after deleting sites with $p_i > d_i$), dispatch times p_i , weights w_i

Ensure: a feasible on-time subset $kept$ (not necessarily optimal)

```
1:  $kept \leftarrow \emptyset$ ;  $total \leftarrow 0$ 
2: for  $i = 1, \dots, n$  do
3:    $kept \leftarrow kept \cup \{i\}$ ;  $total \leftarrow total + p_i$ 
4:   while  $total > d_i$  and  $kept \neq \emptyset$  do
5:      $j \leftarrow \arg \min_{j' \in kept} w_{j'} / p_{j'}$  ▷ smallest weight-to-time ratio
6:      $kept \leftarrow kept \setminus \{j\}$ ;  $total \leftarrow total - p_j$ 
7:   end while
8: end for
9: return  $kept$ , dispatched in increasing-deadline order
```

Proposition 10 (Weighted greedy repair has unbounded worst-case ratio) *For every integer $k \geq 2$ the two-site instance $p = (1, k)$, $d = (1, k)$, $w = (2, 2k - 1)$ satisfies Assumption 1 and the weighted greedy repair of Algorithm 4 returns on-time weight 2, while $W^* = 2k - 1$; the ratio $2/(2k - 1) \rightarrow 0$.*

Proof The algorithm keeps site 1 (total $1 < d_1$). On adding site 2 the total becomes $1 + k > d_2 = k$, so it discards the kept site of smallest ratio w_j/p_j ; site 1 has ratio 2 and site 2 has ratio $(2k - 1)/k < 2$, so site 2 is discarded and the result is $\{1\}$ with weight 2. Site 2 alone completes at $k \leq d_2$, so $W^* \geq 2k - 1$, and the two sites cannot both be on time, so $W^* = 2k - 1$ for $k \geq 2$. $\square$

These two propositions are elementary two-site examples and we offer them as illustrations, not as deep results; on random instances the greedy repair is far better than its worst case (Section 5.2). They do say that some form of reconsideration governed by the weights *and* the dispatch times is needed for any worst-case guarantee. They do not say that the dynamic program or the FPTAS is the only way to obtain one; other polynomial algorithms with guarantees may exist, and we do not claim otherwise.

4.6 Further structural results

Tightness of the FPTAS guarantee.

Theorem 7 guarantees the constant $\rho_{n,\epsilon}$, which exceeds both the classical $1 - \epsilon$ and that Example 1 already previewed at a modest scale. Section 5 confirms the same qualitative gap persists, if less extremely, on random instances of realistic size $1/(1 + \epsilon)$. The next result shows that it cannot be improved for Algorithm 2 as stated.

Proposition 11 (The refined guarantee is asymptotically tight) *Let $\epsilon \in (0, 1)$, $n \geq 2$, and $y = n/\epsilon$, $N = \lfloor y \rfloor$, $\varphi = y - N$. For every $\eta > 0$ there is an instance with n sites satisfying Assumption 1 on which Algorithm 2 returns a set of weight at most $(\rho_{n,\epsilon} + \eta)W^*$. Hence the*

constant $\rho_{n,\epsilon}$ of Theorem 7 is best possible, and it tends to $1/(1+\epsilon)$ as $n \rightarrow \infty$; the classical guarantee $1-\epsilon$ is not attained (it is loose by the factor $1/(1-\epsilon^2)$ in the limit).

Proof Let M be a large integer and $K = \epsilon M/n$ (so $K \rightarrow \infty$ with M). *Family 1* (the term $\frac{n-1}{y}$): $p_i = 1$, $d_i = n$ for all i , $w_1 = M$ and $w_i = \lceil K \rceil - 1$ for $i \geq 2$. All sites fit, so $W^* = M + (n-1)(\lceil K \rceil - 1)$; site 1 has scaled weight N and all the others scaled weight 0, so the algorithm returns $\{1\}$ (zero-scaled sites are never selected) and the ratio $M/W^* \rightarrow (1 + (n-1)\epsilon/n)^{-1} = (1 + (n-1)/y)^{-1}$. *Family 2* (the term $\frac{n-2+\varphi}{N}$): a site b with $(p, d, w) = (1, 1, \lceil KN \rceil)$, a site h with $(2, 2, M)$, and $n-2$ sites z_j with $(1, n, \lceil K \rceil - 1)$. Then $w_{\max} = M = Ky$, so $w'_b = w'_h = N$ and $w'_z = 0$; b and h are incompatible ($1+2 > 2$), so the maximal scaled value is N and the minimal time 1 is attained only by $\{b\}$: the algorithm returns $\{b\}$, of weight $\lceil KN \rceil$, whereas $\{h\} \cup \{z_j\}$ is feasible (h completes at 2, the z_j at $3, \dots, n$), so $W^* \geq M + (n-2)(\lceil K \rceil - 1)$. The ratio is at most $\lceil KN \rceil / (M + (n-2)(\lceil K \rceil - 1)) \rightarrow N/(y + n - 2) = (1 + (n-2+\varphi)/N)^{-1}$. The smaller of the two limits is $\rho_{n,\epsilon}$. $\square$

The constant concerns Algorithm 2 *without* the completion step; with the step the guarantee can only improve, and we do not determine its exact value. Rescaling is also available: running the scheme with $\epsilon' = \delta/(1-\delta)$ guarantees $1-\delta$ for $\delta < \frac{1}{2}$, so the refinement changes constants, not the dependence on $1/\epsilon$.

Family 1 is the extremal one whenever $N(1-\varphi) \geq \varphi(n-2+\varphi)$, in particular when ϵ divides n ; Family 2 is needed otherwise (for $n=3$, $\epsilon=10/13$ the two limits are 0.661 and 0.612). The optional completion step of Remark 4 repairs Family 1 completely, which is why we report both variants in Section 5.3.

Which heterogeneity causes hardness?

Theorems 3 and 8 and Moore's algorithm together settle the complexity of every restriction obtained by making some of the three data vectors constant. For $T \subseteq \{p, w, d\}$ let $\mathcal{C}_T$ be the class of MWHED instances in which each vector named in T is constant ($p_i = p$, $w_i = w$, $d_i = D$ for all i).

Theorem 12 (Classification by homogeneity) *Let $T \subseteq \{p, w, d\}$.*

- (a) *If $p \in T$ or $w \in T$, MWHED restricted to $\mathcal{C}_T$ is solvable in $O(n \log n)$ time.*
- (b) *If $T \subseteq \{d\}$, MWHED restricted to $\mathcal{C}_T$ is weakly NP-hard; in particular it is NP-hard for $T = \{d\}$.*

Thus, assuming $P \neq NP$, an MWHED restriction is polynomial exactly when the dispatch times or the weights are constant, and constancy of the deadlines never helps.

Proof (a) If $p \in T$, apply Theorem 8 with $m=1$. If $w \in T$, all sites have the same weight w , so maximizing on-time weight means maximizing the *number* of on-time sites, which is the problem $1 \parallel \sum U_j$ solved in $O(n \log n)$ time by Moore's algorithm (Moore 1968). (b) $\mathcal{C}_{\{d\}} \supseteq$ the instances of Theorem 3, so it is NP-hard, and it is *weakly* NP-hard in the usual sense (NP-hard, yet solvable in pseudo-polynomial time by Theorem 5); $\mathcal{C}_\emptyset$ contains $\mathcal{C}_{\{d\}}$. $\square$

The classification is an elementary corollary of Theorems 3 and 8 and of Moore’s algorithm; its value is interpretive. It is also not the natural tractability boundary in the scheduling literature, where Moore–Hodgson’s rule is known to extend to agreeable (oppositely ordered) weights and dispatch times (Lawler 1976), a class containing both constant p and constant w . Hardness needs *two independent* sources of heterogeneity, in the dispatch time *and* in the criticality, which compete for the shared time budget; heterogeneity in the hazard deadlines, the part of the instance produced by the hazard model, plays no role. In network terms: if every site is equally far from the depot, or every site is equally critical, the dispatch problem is easy however the hazard spreads. Under the arrival reading the same holds with “constant hazard-arrival time” in place of “constant deadline” (Proposition 4).

5 Numerical study

~~We illustrate Theorems 5 and 7 and Proposition 9 in five complementary experiments: an accuracy comparison across four algorithms on synthetic instances; a runtime comparison isolating the pseudo-polynomial~~ (The study has four purposes: to check every algorithm against an independent exhaustive search (Section 5.1); to compare the algorithms and the heuristics of Section 4 across instance sizes and generators, and to show, in a regime where the FPTAS’s rounding is genuinely active, how its accuracy and cost behave as ϵ shrinks (Sections 5.2–5.6); and to illustrate the model on a real wildfire, with a sensitivity analysis over its uncertain inputs (Section 5.7). All code is self-contained and deterministic given its seeds (see the Data Availability statement); every table below is regenerated by `python3 experiment.py` (about a minute) and `python3 case_study_campfire.py` (about twenty seconds); timings depend on the machine, and all other entries do not.

5.1 Verification against exhaustive search

Before any performance claim we verify the implementations (`verify_small.py`). (i) On 400 random instances with $n \leq 7$ the dynamic program’s optimum equals the optimum over *all* $n!$ dispatch orders, which checks Lemma 2 and Theorem 5 ~~) versus truly polynomial (Theorem 7) dependence on the instance’s numerical magnitude; a sensitivity sweep over~~ from the definition. (ii) On 1,500 random instances with $n \leq 14$ (a third of them with weights up to 10^6 and all with individually infeasible sites left in) the FPTAS’s accuracy parameter ϵ ; a robustness check repeating the accuracy comparison under two alternative synthetic instance-generation regimes; and dynamic program equals the best feasible subset found by enumeration, and the FPTAS returns a feasible set of weight at least $(1 - \epsilon)W^*$ for $\epsilon \in \{0.5, 0.2, 0.05\}$, with and without the completion step, with no violation in 4,500 runs (2,923 of which had active scaling). (iii) The equal-cost greedy of Algorithm 3 equals the dynamic program on 300 instances for $m = 1$, and equals exhaustive search over all assignments to $m = 2, 3$ vehicles on the instances with $n \leq 7$. (iv) Proposition 10’s ratio $2/(2k - 1)$ is reproduced for $k = 3, 10, 100$. (v) The results added in the revision are checked by `verify_extensions.py`: the slot algorithm of Theorem 8 for vehicles with different dispatch times agrees with the counting criterion and with exhaustive search over all

assignments and orders on 1,600 instances ($n \leq 8$, $m \leq 3$), with the matroid axioms on all pairs of feasible sets (on the 1,199 instances with $n \leq 6$), and with a naive greedy on 400 larger instances; the reduction of Proposition 4 agrees with a Partition oracle on 600 instances (344 yes, 256 no); the bound $\rho_{n,\epsilon}$ of Theorem 7 holds on 4,400 random instances, on a local-search adversary (9,000 evaluations) and on both families of Proposition 11, with no violation; and the recursions of Lemma 6 and (3), the tie handling of Lemma 2 and the Partition preprocessing of Theorem 3 agree with brute force cell by cell (800, finally, a small instance built directly from public records of a real wildfire (Section 5.7) 800, to confirm the synthetic findings are not an artifact of how the synthetic instances are generated 600 and 700 instances); the network realisation of Remark 2 is checked with shortest paths on 117 Partition instances. The statements of Sections 3 and 4 are in addition machine-checked in Lean 4 (Data Availability).

5.2 Accuracy across instance sizes

For $n \in \{10, 15, 20, 25, 30, 50, 75, 100\}$ we generate 20 random instances each (p_i uniform on $\{1, \dots, 20\}$, w_i uniform on $\{1, \dots, 100\}$, d_i uniform on $\{1, \dots, \sum_i p_i\}$); and compare four algorithms, individually infeasible sites deleted as in Assumption 1) and compare: the true optimum via (Theorem 5's exact dynamic program); the FPTAS of Theorem 7 at $\epsilon \in \{0.1, 0.2\}$, implemented exactly at $\epsilon \in \{0.1, 0.2\}$, implemented as in the proof (the value-indexed dual DP, not a shortcut); a weighted Moore-Hodgson-style greedy repair heuristic (dispatch in earliest-deadline order, and whenever the running schedule becomes infeasible, repeatedly drop the scheduled site with the smallest weight-to-time ratio until feasible again—a natural, always-feasible heuristic generalizing Moore's unweighted rule, for which we claim no approximation guarantee; the weighted greedy repair (Algorithm 4); and the naive baseline of two deadline-order baselines, naive EDD and EDD with skipping (Proposition 9 (dispatch in earliest-deadline order, simply keep whichever sites happen to still be on time, with no repair at all)). Naive EDD keeps dispatching to sites that are already late, so it is a weak anchor; EDD with skipping is the fairer simple baseline. Table 4 reports the ratio of each method's on-time weight to the true optimum, averaged over the 20 instances at each size. Algorithm 4 states the greedy repair heuristic precisely; feasibility of its output is guaranteed by the same inductive argument used for the unweighted Moore-Hodgson algorithm (each repair step can only shorten, never lengthen, the completion time of every remaining scheduled site), but—unlike Algorithms 1–3—we prove no approximation ratio for it; it is included as a realistic, easy-to-implement point of comparison, not as a fifth theorem mean ratios to the optimum, and Figure 2 plots them.

sites $1, \dots, n$ with $d_1 \leq \dots \leq d_n$, dispatch times p_i , weights w_i a feasible on-time subset $kept$ (not necessarily optimal) $kept \leftarrow \emptyset$; $total \leftarrow 0$ $kept \leftarrow kept \cup \{i\}$; $total \leftarrow total + p_i$ $j \leftarrow \arg \min_{j' \in kept} w_{j'}/p_{j'}$ $kept \leftarrow kept \setminus \{j\}$; $total \leftarrow total - p_j$ $kept$, dispatched in increasing-deadline order

At both approximation levels tested, the The FPTAS recovers the true optimum on essentially every one of the 160 random instances (mean ratio rounds to 1.000 throughout), far outperforming its optimum on every instance, but this is *not* evidence

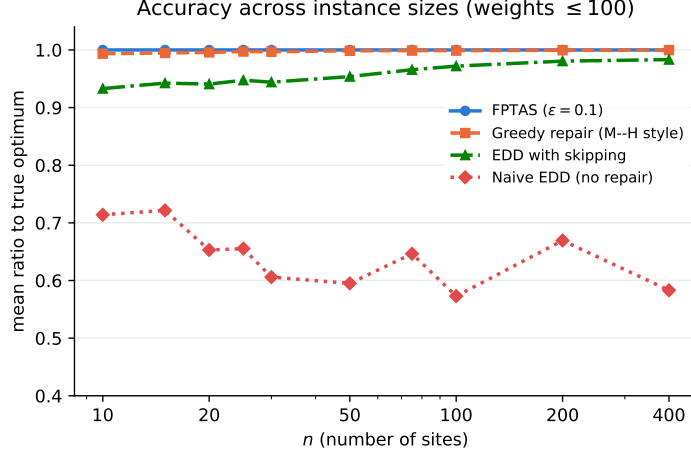


Fig. 2 Table 4 plotted. The FPTAS and greedy repair sit at 1.0; EDD with skipping improves slowly with n ; naive EDD fluctuates around 0.6 with no trend and, on individual instances, captures only a few percent of the optimum.

about the approximation scheme. With weights at most 100 the scaling factor $K = \max(1, \epsilon w_{\max}/n)$ equals 1 for $\epsilon = 0.1$ at every $n \geq 15$ and for $\epsilon = 0.2$ at every $n \geq 25$ (last row of Table 4), so no rounding takes place and the FPTAS coincides with the exact value-indexed dynamic program; a ratio of 1.000 is then guaranteed, not informative. The next subsection therefore uses weights large enough that the rounding is active. Among the heuristics, the greedy repair captures 99.3–100% of the optimum on average but has a worst ratio of 0.93 at $n = 10$, consistent with its having no guarantee (Proposition 10); naive EDD shows no trend with n , as a rule that never reconsiders would not, though a worst-case $(1 - \epsilon)$ -guarantee, consistent with the well-documented empirical looseness of knapsack-style FPTAS result such as Proposition 9 does not by itself predict average behavior on random instances.

5.3 The FPTAS with active scaling

We now take weights up to 10^6 with $p_i \leq 20$, so that the time-indexed exact dynamic program stays cheap (its table depends on P , not on the weights) and serves as the reference, while the FPTAS’s scale factor K is in the hundreds to thousands. Three families are used, with $n = 100$ and 30 instances per ϵ for the first two. *Random*: weights uniform on $\{1, \dots, 10^6\}$. *Strongly correlated*: a common deadline and $w_i = 1000p_i + 500$ with p_i up to 1000, the hard family for knapsack-type schemes (Hejl et al. 2022 study this regime for exact algorithms). *Adversarial*: the first family of Proposition 11, with one dominant site and $n - 1$ sites just below the scaling granule. This column is a single deterministic instance, not an empirical worst case; its entries equal the closed form $n/(n + (n - 1)\epsilon)$ of the proposition for $n = 100$ and large M . For each, Table 5 reports the worst ratio over the instances for Algorithm 2 as stated and, for the adversarial family, with the completion step of Remark 4.

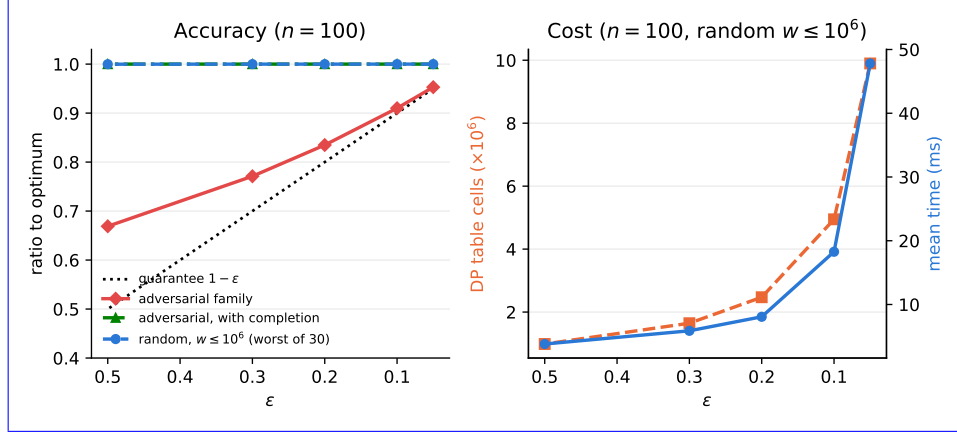


Fig. 3 Tables 5 and 6 plotted. Left: the worst ratio on the adversarial family tracks the guarantee $1 - \epsilon$ (it stays above it and, as Proposition 11 shows, approaches $1/(1 + \epsilon)$); on random instances the FPTAS is essentially exact, and the completion step makes the adversarial family exact. Right: cost grows in proportion to $1/\epsilon$, as the $O(n^3/\epsilon)$ bound says.

Two things are visible. First, on random and on strongly correlated instances the FPTAS is nearly exact even at $\epsilon = 0.5$ (worst ratio 0.9993), far above its guarantee: the bound $\epsilon w_{\max}$ charges every selected site a full rounding error K and compares the total to $w_{\max}$, whereas on these instances the optimum contains many sites and $W^* \gg w_{\max}$. This is why the ratio is close to 1, and it is the answer to the question of why accuracy is near 1 in all earlier tables. Second, the guarantee is *not* vacuous: on the adversarial family, which Proposition 11 shows is the worst case, the ratio falls to 0.669 at $\epsilon = 0.5$ and follows the guarantee as ϵ shrinks, so a planner who needs a worst-case bounds (Ibarra and Kim 1975): the scaling construction is designed to guarantee the worst case, not to describe the typical one. The greedy repair heuristic, despite carrying no proven guarantee, is a substantial practical improvement over doing nothing—capturing 99.3–99.9% of the optimum—because reconsidering even a single locally-worst commitment when infeasibility first arises already recovers most of the value a full optimization would. The naive EDD-baseline, which never reconsiders anything, captures only 32–62% of the optimal on-time weight and shows no trend of improving with instance size, exactly as Proposition 9 predicts: its worst-case ratio does not shrink with n , it is simply unbounded below on adversarial instances and highly variable on random ones. The inexpensive completion step removes this failure on the family. Cost grows in proportion to $1/\epsilon$ (Table 6), as predicted, and is independent of P (next subsection).

5.4 Runtime: time-indexed versus value-indexed

Theorem 5's exact DP dynamic program runs in $O(nP)$ time, where $P = \sum_i p_i$ can be with $P = \sum_i p_i$ exponential in the input's bit length; Theorem 7's FPTAS runs in time polynomial in n and $1/\epsilon$, independent of the FPTAS's table does not depend on P . To see this gap empirically rather than only asymptotically, we fix $n = 40$ and, $\epsilon = 0.1$ and, draw weights up to 10^6 (so the scaling is active at every setting), vary

the range $p_{\max}$ from which ~~processing-dispatch~~ times are drawn (so P grows roughly linearly with $p_{\max}$), ~~timing~~, and ~~time~~ both algorithms on ~~5-5~~ random instances per setting. ~~Table 7 reports the mean wall-clock time.~~

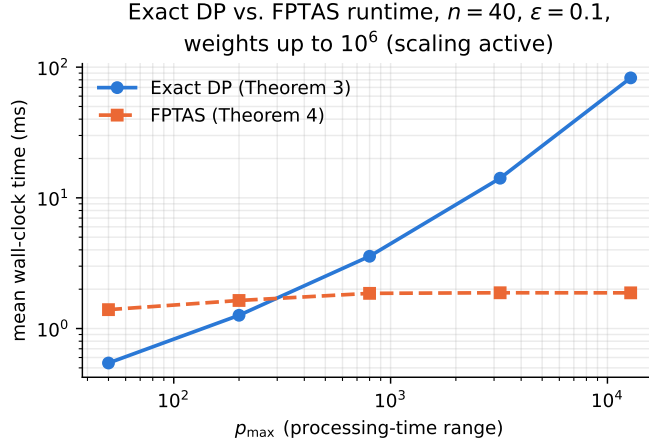


Fig. 4 Table 7 on log-log axes: the exact DP’s time grows with $p_{\max}$ (roughly linearly or slightly faster once the fixed overhead is exceeded), while the FPTAS’s stays flat.

The exact DP’s runtime grows essentially linearly with $p_{\max}$, as its $O(nP)$ bound predicts, increasing roughly 240-fold as $p_{\max}$ increases grows 256-fold; the FPTAS’s runtime stays flat, within measurement noise, across the same range, since its table size $O(n^3/\epsilon)$ never depends on P at all (the smallest settings are dominated by fixed overhead), whereas the FPTAS is flat up to noise. At the smallest $p_{\max}$ tested here the exact DP is actually the exact program is faster in absolute terms, since its (pseudo-polynomial but small) table is cheaper than the FPTAS’s fixed $O(n^3/\epsilon)$ overhead—a reminder that an FPTAS’s asymptotic advantage only pays off once the instance’s approximation scheme pays off only when the numerical magnitude is large enough, exactly the regime Theorem 3’s weak-NP-hardness result says the exact DP cannot handle efficiently. We stress that this comparison is between a time-indexed and a value-indexed table, and that the dependence on the weights has simply moved: a value-indexed *exact* program would depend on $\sum_i w_i$ in the same way.

5.5 Comparison with a general-purpose solver

Theorem 7’s table has $O(n^3/\epsilon)$ entries, so shrinking ϵ should, in principle, grow both the accuracy and the running time of the FPTAS. We fix $n = 30$ and sweep $\epsilon \in \{0.5, 0.3, 0.2, 0.1, 0.05, 0.01\}$ on the same 20 random instances at each value, reporting the mean ratio to the true optimum. As an independent baseline we solve the same instances with a general-purpose mixed-integer solver (HiGHS, through `scipy.optimize.milp`) on the subset formulation: binary x_i , sites in

deadline order, and ~~the mean wall-clock time in Table ??~~ for every i the constraint $\sum_{j \leq i} p_j x_j + (P - d_i)x_i \leq P$, which by Lemma 2 expresses feasibility, maximizing $\sum_i w_i x_i$. On 10 random instances per size ($p_i \leq 20$, $w_i \leq 100$) both methods return the same optimum on every instance, and the dynamic program is much cheaper: Table 8. The comparison is not meant to dismiss MIP technology; it shows that for this structure the dynamic program of Theorem 5 is the right tool when dispatch times are small integers, and it provides an independent check of the optima.

~~Even at the loosest setting tested, $\epsilon = 0.5$ (a 50% worst-case loss allowed), the FPTAS already recovers 99.99% of the optimum on average, and by $\epsilon = 0.3$ it matches the optimum on every instance in this sample; consistent with Example 4 and Section 5's accuracy table, the worst-case bound $(1 - \epsilon)W^*$ is a safety margin that is rarely approached in practice, not a description of typical behavior. The runtime grows only mildly over this range at $n = 30$ (a two-order-of-magnitude shrinkage in ϵ , from 0.5 to 0.01, produces roughly a 50% increase in running time, not the $50\times$ the $O(n^3/\epsilon)$ bound alone would suggest as an upper bound), because the table's row dimension is capped by $\sum_i w'_i$, which grows sub-linearly in $1/\epsilon$ once the scaled weights $w'_i = \lfloor w_i/K \rfloor$ saturate at small integer values; the $O(n^3/\epsilon)$ bound is a worst-case ceiling; on these instances the effective table is considerably smaller.~~

5.6 Robustness to the instance-generation regime

~~The accuracy comparison of Section 5.1 uses uniformly distributed weights and a deadline horizon spanning the full range $[1, \sum_i p_i]$. To check that its qualitative pattern the pattern of Table 4 is not an artifact of that specific choice, the generator we repeat the comparison at $n = 30$ under two different regimes: (20 instances each) under heavy-tailed weights (drawn from an exponential distribution rather than uniform, so a few sites are far more critical than the rest exponential, mean 20) and tight deadlines (the deadline horizon narrowed to $0.5 \sum_i p_i$, so on average only about half the sites can be served regardless of order). Table 9 reports the same three ratios as before under each regime. $0.5 \sum_i p_i$.~~

~~The FPTAS and the greedy repair heuristic are essentially unaffected: both continue to recover almost all of the optimum, confirming that Section 5.1's pattern is not an artifact of uniform weights or a loose deadline horizon. The naive EDD baseline, by contrast, gets worse under both regimes—down to 2.6% of optimum under tight deadlines—which is exactly what Proposition 9 predicts: tightening deadlines or increasing weight heterogeneity both widen the gap between the single early commitment naive EDD makes and the value it forecloses, since greedy repair is essentially unaffected and EDD with skipping degrades under tight deadlines, where there is less slack to absorb a poor early choice and more value concentrated in whichever sites get displaced by it. commitment; naive EDD collapses to 12.5% of the optimum, largely because it also wastes time on sites that are already late. (As in Section 5.2, the FPTAS entries are 1.000 partly because the scaling is vacuous at weights ≤ 100 .)~~

5.7 A real-world illustration: the 2018 Camp Fire

The instances above are synthetic by ~~design—MWHED is a new problem with no~~ existing design: MWHED has no benchmark set, and the ~~robustness check just given~~ specifically needs controllable distributional properties that only a generator, not a single fixed dataset, can provide experiments need controllable distributions. To ground the model’s scale ~~and behavior~~ in a real event, ~~we also~~ we build one small instance ~~directly~~ from public records of the 2018 Camp Fire (Butte County, California), the deadliest wildfire in California history (85 ~~fatalities, over~~ civilian fatalities and 18,000 ~~structures destroyed~~ 804 structures destroyed, according to the NIST reconstruction cited below). We ~~emphasize which numbers below are real measurements and which are disclosed modeling assumptions, rather than blending the two silently~~ separate real, documented inputs from disclosed modelling assumptions, and we present the case as an *illustration of the model and of its sensitivity*, not as an operational recommendation or a reconstruction of what any crew could have done.

Real Documented inputs. The fire’s ignition point (a PG&E transmission tower near Pulga, CA) and its progression timeline—in particular, fire reaching Concow approximately 52 minutes after ignition, and spot fires igniting in Paradise approximately 71 minutes after ignition—are as documented in a Every hazard-arrival time is a clock time documented in the National Institute of Standards and Technology forensic reconstruction (Maranghides et al. 2021). We take four real, named ~~communities threatened by the fire~~ (NIST) reconstruction of the fire’s progression (Maranghides et al. 2021); none is extrapolated. For each of four real communities in the fire’s path we use the earliest documented event of the kind NIST reports for it: the first structures burning and spot fires igniting in *Concow* (07:25 on 8 November), the first spot fires arriving in *Paradise* (*Concow, Paradise, Magalia*, 07:44; the detailed sections of the report put the first 911 call about a Paradise spot fire at 07:49, which lies inside the sensitivity range below), spot fires in Old Magalia, the earliest-affected part of *Magalia* (08:40; most of Magalia burned much later), and the fire running through *Yankee Hill*, documented as “well established” at 08:40 on 9 November, about a day after ignition and therefore an almost unconstraining deadline. The events are therefore not identical across communities, which is a limitation of the data, not of the model; the documentation is also uneven across communities, and Yankee Hill, ~~Butte County~~ with their real geographic coordinates and’s event is an upper bound on the fire’s arrival there (the instance is in effect a three-site one, as Yankee Hill’s deadline never binds). The weights are the 2010 U.S. Census populations (the ~~census immediately preceding the fire, used as a proxy criticality weight w_i~~ last count before the fire; later counts reflect the fire’s aftermath, not the population at risk). A real ~~regional fire-agency aftermath, and the NIST report quotes different, survey-based figures~~: Paradise 26,218, Magalia 11,310, Concow 710, Yankee Hill 333. Community coordinates are their published centroids, and the depot (Oroville, CA) ~~anchors the dispatch side~~ is a modelling choice.

Disclosed modeling modelling assumptions. Two quantities are not directly measured. First, ~~(i) p_i is computed from the real~~ twice the great-circle distance between the depot and ~~each site (a real site i (a lower bound on true road distance))~~ divided by an assumed average response speed; we report ~~results at two~~

illustrative speeds, 50 km/h (conservative, winding mountain roads) and 80 km/h (an optimistic highway-speed upper bound). Second, Magalia and Yankee Hill have no directly documented fire-arrival timestamp in the source we use, so their deadlines d_i are extrapolated: we compute the real fire-spread rate implied by Concow’s and Paradise’s real distance from ignition and real documented arrival time (12.1 and 14.2 km/h respectively, averaging 13.1 km/h), then apply this averaged rate to Magalia’s and Yankee Hill’s own real distance from the ignition point. Table 10 reports the resulting instance and Table 11 the outcome of running Algorithms 1–4 unmodified on it (code and full source citations: `case-study-campfire.py` optimistic), and Table 12 varies the road factor. (ii) Time zero is the initial dispatch time in NIST’s dispatch log, 06:31 on 8 November (NIST’s own estimate of ignition is about 06:20, and the first 911 call is at 06:25); the crew leaves at time zero (the dispatch to the fire’s origin, not a decision to defend these communities, so this is optimistic), and the sensitivity analysis adds a departure delay, which only postpones it. (iii) The *arrival reading* of Remark 1 is used: a site is *protected* if the crew *arrives* before the documented event, so the MWHED deadline is $d_i = d_i^{\text{haz}} + p_i/2$. “Protected” means arrival only: the model has no on-site service time (which would simply add to p_i) and no notion of what protection achieves. Table 10 gives the instance.

At the conservative Results. At 80 km/h the trips are short enough that every algorithm, even naive deadline order, serves all four communities. At 50 km/h ~~response speed, Concow~~ the picture changes: Concow’s early deadline (86 minutes) and Paradise’s (108) cannot both be met, and the optimum gives up Concow (weight 710) to protect Paradise, Magalia and Yankee Hill, for 37,861 of the 38,571 total. Deadline-order rules go to Concow first because its deadline is earliest, and they pay for it: naive EDD reaches an on-time weight of only 1,043, and EDD with skipping 12,353, a third of the optimum, because serving Concow makes Paradise late. The greedy repair finds the optimum on both speeds, and the FPTAS with $\epsilon = 0.1$ returns 99.1% of the optimum at 50 km/h: Yankee Hill’s weight 333 is below the scaling granule $K \approx 655$ and is dropped, which is exactly the zero-scaled-site loss of Remark 4 (re-inserting it recovers the optimum). Under the stricter *return reading* (the crew must be back before the fire arrives) the answer changes: at 50 km/h Concow is individually infeasible (round trip of 63 minutes against an arrival minute of 54), the optimum is Paradise and Yankee Hill (26,551), and Magalia ~~are individually infeasible—even dispatched first and alone, a crew from the Oroville depot cannot reach them before the fire does, a sobering but real reflection of how far this fire’s actual spread outran any single-vehicle response. The optimal policy recognizes this immediately and sends the crew directly to Paradise, saving all 26,218 people at risk there; naive earliest-deadline dispatch, recall, tries Concow and Magalia first because their deadlines are earlier, exhausts the time budget on two sites it can never reach, and arrives at Paradise too late to help anyone—an on-the-nose illustration of Proposition 9’s unbounded worst-case loss, not a constructed adversarial example. At the more optimistic 80’s 11,310 is lost; this is why Remark 1 asks the user to state the reading.~~

The round-trip premise on this geography. The four communities are 4–15 km apart, against depot distances of 22–37 km, so the depot-spoke model is far from the geometry. By Proposition 1 it is conservative; Table 12 measures by how

The 2018 Camp Fire case study: geography and hazard times
(marker size $\propto$ 2010 census population)

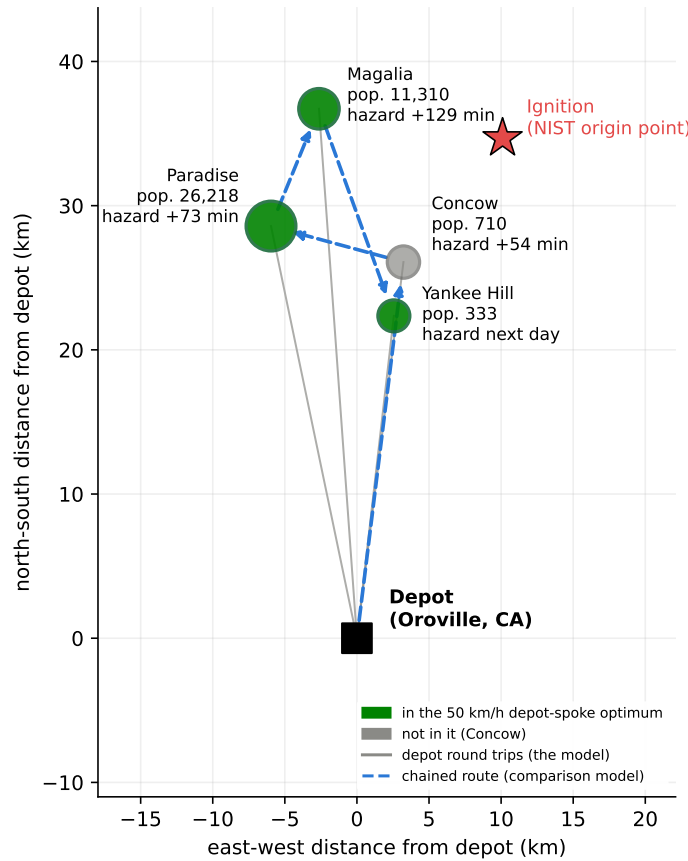


Fig. 5 The geography behind Table 10: the depot, the fire's origin point as given by NIST, and the four communities, plotted from their published coordinates, with the hazard time of each. Solid gray lines are the depot round trips of the MWHED model; the dashed arrows are the chained route of Table 12. Green markers are the sites of the 50 km/h depot-spoke optimum of Table 11.

much, comparing the spoke optimum with the optimum of a chained route (direct travel between consecutive sites, great-circle distance times a road factor ρ , exhaustive over all ordered subsets). With $\rho = 1$ the chained route protects all four communities at both speeds; at 50 km/h ~~speed all four sites are individually reachable, yet the optimal policy~~ the spoke model loses Concow, weight 710. With road factors 1.3 and 1.6 the spoke model loses more, because it charges ρ on every round trip while the hazard times stay fixed (Paradise remains in the optimum), whereas the chained route still protects all four. For this event the depot-spoke optimum is therefore a safe but pessimistic plan, and the structural findings below should be read as findings about the ~~still model goes straight to Paradise alone rather than visiting Concow~~, to be confirmed on a routing model before any operational use.

Sensitivity to the uncertain inputs. The inputs are uncertain in several ways: the response speed, the documented event times (each is an estimate from observations, and the events differ in kind across communities), and the dispatch delay. We draw 20,000 scenarios with the speed uniform on $[40, 90]$ km/h, the event times of Concow and Paradise scaled by independent factors uniform on $[0.85, 1.15]$, those of Magalia and Yankee Hill (whose events are less sharply defined) on $[0.70, 1.30]$, and a departure delay uniform on $[0, 20]$ minutes, and solve each exactly. We also run a *correlated* variant in which one common factor, uniform on $[0.70, 1.30]$, scales all event times (a systematic bias of the fire reconstruction), with a small independent factor on $[0.95, 1.05]$ per site. Standard errors of the reported shares are at most 0.4 percentage points.

Protocol. A plan is a fixed visiting sequence in the order Concow, Paradise, Magalia, Yankee Hill, restricted to its sites; every visit is a round trip, and a site that Assumption 1 deletes in a draw is skipped (so a hopeless first visit does not consume time). The *retained share* of a plan in a draw is its protected weight divided by the optimum of that draw. If instead every site of the plan is visited even when hopeless, the mean retained share of the nominal plan drops from 0.319 to 0.289; we report the first protocol.

Paradise is in the optimum in all draws of the independent variant and in 99.2% of the correlated one; at 50 km/h it leaves the optimum only if its weight falls below 710 residents and misses Paradise entirely. Both the exact DP and the FPTAS recover the true optimum in every case, Concow's. The decision is in effect a single binary conflict, whether to visit Concow (710) before Paradise (26,218); Yankee Hill's event is a day away and never binds, and Magalia rarely does. The optimal set is Paradise, Magalia and Yankee Hill in 63.1% of the independent draws, Paradise with Yankee Hill in 20.7% and all four communities in 16.2%.

The nominal plan depends on the nominal speed. The optimum of the nominal instance (no delay) includes Concow only from about 68 km/h upward, since the round trips to Concow and Paradise must both fit before Paradise's deadline. The plan that is optimal at 80 km/h, an optimistic corner of the speed range, visits Concow first and is fully on time in only 16.2% of the draws, retaining on average 31.9% of the drawn optimum (fifth percentile 2.8%), because a delay or a slower road lets the Concow visit make Paradise late; Paradise has only 12 minutes of slack. The plans that are optimal at 65 or 50 km/h, and the greedy repair heuristic matches them here as well. This single real instance cannot substitute for the synthetic sweep's statistical evidence, but it shows the same qualitative pattern is not a synthetic-data artifact, and the 50-sample-average plan described next, all drop Concow: Paradise, Magalia and Yankee Hill. That plan is fully on time in 79.3% of the draws and retains 99.7% of the drawn optimum on average (fifth percentile 98.2%); since Concow carries 1.8% of the total weight, any plan that drops it retains at least 98.2% of an optimum that includes it. The *sample-average plan* is the subset (in the same visiting order) with the largest average protected weight over an independent sample of 2,000 draws; it coincides with the conservative-speed optimum, so here the scenario sample confirms what a conservative nominal speed already gives, and its value would lie in problems where the right nominal is not obvious. Under correlated errors the figures are 26.0%

and 37.2% for the 80 km/h scenario’s already infeasible sites are exactly the kind of situation Section 6.1’s discussion of multiple vehicles is motivated by: a single depot’s single crew has a hard limit on what it can save, regardless of how it is scheduled plan and 79.1% and 99.5% for the others. Finally, a chained route with road factor 1.0 (resp. 1.3) protects all four communities in 96.1% (resp. 79.5%) of the independent draws.

What the case study does and does not show. The finding is conditional and modest: in this model the exact optimum inherits whatever nominal speed (that is, slack) is assumed. When the assumption is optimistic, the plan spends Paradise’s slack on a small community and fails under timing errors; planning for a conservative speed removes the failure at a cost of at most the small community’s weight. This is a property of the depot-spoke model, which Table 12 shows to be a poor fit for this geography, and a scenario formulation (Section 6.3) is the natural way to make such choices. The study does not show what a real crew should have done: the model ignores on-site service, road closures, evacuation (the main protection in this event) and the fact that the dispatch itself changes the fire. The objective is also a choice with ethical content. Weighting by population and conceding the sites with the least weight is utilitarian; a planner may instead require a minimum protection level for every community, or weight by vulnerability. A set of sites that must be protected can be forced into the plan, within the same model, by adding a constant larger than the total weight to their weights (provided each is individually feasible); proportional-fairness constraints generally break the knapsack structure. We therefore treat the numbers above as a worked illustration of the model and its sensitivity, on a single event with four sites, and as nothing more.

6 Discussion and extensions

The results of Section 4 settle treats the single-depot, single-vehicle, deterministic-deadline case completely. Three natural directions relax one assumption at a time; we discuss, for each, what carries over from our analysis and what specifically breaks, rather than simply listing them as unexplored future work.

6.1 Multiple depots or vehicles

Suppose $m > 1$ vehicles are available, each able to serve a disjoint subset of sites, still under the same round-trip dispatch time and deadline structure. Call this serving its own sequence of round trips, as in MWHED- m (MWHED itself is MWHED-1). Two facts are immediate. First, of Section 4.4. Four facts describe what changes.

Hardness persists for every fixed m . Take a common-deadline instance with deadline D and total weight $\Omega = \sum_i w_i$ (the hard instances of Theorem 3) and add $m - 1$ “blocker” sites with $p = d = D$ and weight $\Omega + 1$. A blocker occupies a vehicle for all of $[0, D]$: if anything is served on that vehicle before it, the blocker is late, and if anything is served after it, that site is late. So each served blocker uses a vehicle exclusively, dropping a blocker costs $\Omega + 1$ and can free a vehicle worth at most Ω , and an optimum serves all $m - 1$ blockers and solves the original instance on the one remaining vehicle. Hence MWHED- m is NP-hard for every fixed $m \geq 1$: the $m = 1$ instance embeds into any MWHED- m instance by adding $m - 1$ vehicles with no sites

to serve, so Theorem 3’s hardness persists. Second, (Simply adding idle vehicles to an instance would not do: extra vehicles can serve sites, which changes the problem.)

The exact dynamic program grows with m . Theorem 5’s dynamic program does not extend at the same complexity: the natural generalization tracks a completion-time state per vehicle; recursion extends by tracking a completion time per vehicle. Assign sites in deadline order (Lemma 2, applied on each vehicle) and let $f(i, t_1, \dots, t_m)$ be the maximum on-time weight using sites $1, \dots, i$ when vehicle k has accumulated exactly t_k occupancy; site i may be appended to any vehicle k with $t_k \leq d_i$, giving a table of size $O(nP^m)$ rather than $O(nP)$ —pseudo-polynomial—

$$f(i, t_1, \dots, t_m) = \max \left(f(i-1, t_1, \dots, t_m), \right. \\ \left. \max_{k: p_i \leq t_k \leq d_i} (f(i-1, t_1, \dots, t_k - p_i, \dots, t_m) + w_i) \right).$$

The table has $O(nP^m)$ cells, pseudo-polynomial for fixed m , but with an exponent that grows with the number of vehicles, not merely a constant-factor slowdown. Whether a smarter exact algorithm avoids this blow-up, or whether the m^{th} power is unavoidable, is open; this connects MWHED- m to [Whether this is unavoidable is open here](#); the parallel-machine scheduling literature’s own maximum-weighted-on-time-jobs problem, whose complexity landscape beyond a single machine is [treats the problem as](#) considerably less settled than the single-machine case we resolve here (Lenstra et al. 1977). Recent evidence confirms this: Heeger and Molter (2025) show that even the parallel-machine analogue of our tractable equal-cost special case—identical processing times across identical machines, $P|r_j, p_j=p|\sum w_j U_j$ —is already NP-hard, (Lenstra et al. 1977). We conjecture, but have not verified, that value-scaling extends to an m -dimensional state.

Equal-size sites stay tractable, even for vehicles of different speeds. Theorem 8 solves MWHED- m with equal-size sites in $O(m + n \log n)$ time for every m , identical vehicles or vehicles with different dispatch times p_v . This is the multi-vehicle analogue of the single-vehicle special case, and it holds without release dates.

Release dates are a different problem. Theorem 8 assumes that every vehicle and every site is available from time 0. Heeger and Molter (2025) show that the *unweighted* problem on m identical machines with equal processing times and release dates, $P|r_i, p_i=p|\sum U_i$, is NP-hard and $W[2]$ -hard parameterized by the number of machines, once release dates are allowed. Translated into our setting, this suggests that Theorem 8’s tractability for MWHED-1 is unlikely to survive as cleanly into MWHED- m for $m \geq 2$, even restricted to the equal-dispatch-cost case—the multi-vehicle relaxation appears to cost more than just an exponent in the DP’s table size. We conjecture the FPTAS. Hence the tractability of Theorem 7 extends to MWHED- m for fixed m via the same value-scaling idea applied to an m -dimensional state (the scaled-weight axis is untouched by the number of vehicles), but we have not verified this in detail and do not claim it as a result here.

To see concretely where the P^m term comes from: assign sites to vehicles greedily by deadline as in Lemma 2, but now track, for each vehicle $k = 1, \dots, m$, its own

cumulative dispatch time t_k . The natural generalization of $f(i, t)$ in Theorem 5’s proof becomes $f(i, t_1, \dots, t_m)$: the maximum on-time weight achievable using sites $1, \dots, i$, where vehicle k has accumulated exactly t_k dispatch time. Site i can be assigned to whichever vehicle k currently has $t_k + p_i \leq d_i$, giving the recursion

$$f(i, t_1, \dots, t_m) = \max \left(f(i-1, t_1, \dots, t_m), \right. \\ \left. \max_{k: t_k \geq p_i, t_k \leq d_i} (f(i-1, t_1, \dots, t_k - p_i, \dots, t_m) + w_i) \right),$$

a direct generalization of (3) with m time axes instead of one. Since each axis ranges over $0, \dots, P$, the table has $O(P^m)$ cells per site, $O(nP^m)$ overall—exactly the blow-up claimed above, and the reason a genuinely new idea, not just a bigger table, would be needed to bring MWHED- m ’s exact algorithm back to a complexity independent of m in the exponent⁸ for an arbitrary number of identical vehicles does not carry over to release dates (unless $P = NP$). That result does not concern a single vehicle: for one machine, equal processing times, weights and release dates are polynomial (Baptiste 1999). Release dates have no counterpart in MWHED as defined here, since every vehicle is at the depot at time 0 and every site can be served from the start; they would arise if sites become reachable, or crews available, only at given times (for instance, a road cleared by a firebreak), and extending the model in that direction for several vehicles is a natural open problem.

6.2 A relocating depot or growing-radius hazard

Definition 1 takes each site’s round-trip dispatch time p_i as a fixed constant, appropriate when the depot is stationary. If the depot itself can relocate between dispatches—a mobile command post following a fire line, for instance— p_i becomes a function of the depot’s current position, which in turn depends on the dispatch order already chosen. Lemma 2’s exchange argument relies critically on p_i being order-independent (Section 3’s observation that every visit is a self-contained round trip); with a relocating depot this fails, and the problem acquires genuine routing structure on top of its scheduling structure. We expect this variant to be at least as hard as MWHED (it contains MWHED as the special case of a depot that chooses not to move) and likely strictly harder, since the order in which sites are visited now affects future dispatch costs, not just which deadlines are met. [Proposition 1 bounds the effect from one side: with a fixed depot and metric travel times, allowing the crew to chain sites can only enlarge what it protects \(a hazard that cuts direct roads between sites while the depot spokes stay open would break this\), so the round-trip optimum is a safe lower bound, and Table 12 shows the gap can be large when sites are clustered. A full routing model \(orienteering or deadline-TSP, Section 2\) would remove the gap at the price of losing the exact algorithms and guarantees of this paper.](#)

It is worth being precise about what “growing-radius hazard” does and does not already mean for a *stationary* depot, since the two relaxations named in this subsection’s title are not the same complication. If the hazard simply expands outward from

a fixed origin at a fixed, known rate, its arrival time at each site is already fully captured by the deadline d_i in Definition 1—this is exactly the reduction described in Section 1, and requires no extension to our model at all. The genuine second relaxation, distinct from a relocating depot, arises if the spreading hazard itself degrades the road or access network the depot’s vehicle uses—for instance, a wildfire making a route impassable once it crosses that route, or rising floodwater cutting off a bridge—so that p_i is not merely unknown but a function of *when* site i is reached, since a later attempt may have to detour around hazard-affected infrastructure that an earlier attempt would not have. This again breaks Lemma 2’s order-independence assumption, but through the road network’s degradation rather than the depot’s own movement, and the two effects would compound if both are present simultaneously.

6.3 Uncertain parameters

Definition 1 takes ~~each deadline the dispatch times p_i , the deadlines d_i and the weights w_i~~ as known exactly. ~~If d_i is instead a random variable (as it would be if derived from a stochastic hazard-spread simulation), the natural analogue replaces the hard feasibility indicator $\mathbb{K}[C_i(\sigma) \leq d_i]$ in (1) with $\Pr[C_i(\sigma) \leq d_i]$, turning MWHED into a chance-constrained or expected-value scheduling problem. Lemma 2’s proof does not survive this change unmodified: the exchange argument compares two fixed completion times against two fixed deadlines, and with random deadlines the ordering that maximizes~~ In practice each is an estimate: arrival times come from a fire-spread, flood or plume simulator and carry its forecast error; occupancies depend on road conditions that the hazard itself may degrade; weights are proxies for population or asset exposure. The optimal solutions and the structural statements above are conditional on these inputs, and the case study of Section 5.7 shows that the nominal optimum can be brittle under plausible perturbations. We discuss three ways to model the uncertainty, and which of our results survive each.

Scenario-based (stochastic) models. Let uncertain parameters take values $\xi_s = (p^s, d^s, w^s)$ with probabilities π_s for $s = 1, \dots, S$, and let the dispatch order σ be chosen before the scenario is revealed. The natural objective is the expected on-time weight ~~need not sort sites by any single deterministic statistic of d_i , for the same reason that earliest-due-date sequencing is not generally optimal under stochastic due dates in the classical scheduling literature $\sum_s \pi_s \sum_i w_i^s \mathbb{K}[C_i^s(\sigma) \leq d_i^s]$, which for $S = 1$ is MWHED and is therefore NP-hard. Lemma 2 holds scenario by scenario but not simultaneously: no single order sorts all scenarios’ deadlines, and the exchange argument fails. A small example makes this concrete: two shows that sorting by expected deadline can be nearly twice as bad as the optimum. Two sites with $p = 1$ each, site A with $w_A = 1$ and a deterministic deadline $d_A = 2$ deadline 2, and site B with $w_B = 10$ and a random deadline $d_B = 1$ or $d_B = 100$, each equal to 1 or 100 with probability $\frac{1}{2}$, so $\mathbb{E}[d_B] = 50.5 > \mathbb{E}[d_A] = 2$. Sorting by increasing $\mathbb{E}[d_i]$ dispatches each, so $\mathbb{E}[d_B] = 50.5 > \mathbb{E}[d_A] = 2$. Serving A first A completes at $1 \leq 2$ (always on time), and B completes at 2, on time only when $d_B = 100$, giving expected value $1 + 10 \cdot \frac{1}{2} = 6$. Dispatching (the expected-deadline order) gives $1 + 10 \cdot \frac{1}{2} = 6$; serving B first instead first gives $10 + 1 = 11$, because B completes at 1, on time whenever $d_B \geq 1$, which holds for’s tight realization is avoidable only by~~

acting immediately. This is the setting of two-stage stochastic programming, in which a first-stage plan is evaluated against scenario-dependent recourse; evacuation planning under uncertain demand and network damage (Wang 2020) and facility location under disruptions (Koca 2023) are close relatives in the disaster-response literature. A sample-average formulation of the expected-weight objective is a mixed-integer program whose combinatorial core is MWHED with scenario-dependent deadlines; a Benders-type decomposition by scenario is natural, and exploiting Lemma 2 within a scenario to generate cuts is, we believe, a promising direction that we have not pursued.

Robust feasibility. Feasibility of a set of sites is monotone in the parameters: it is preserved when d_i increases or p_i decreases. A set that is on time for ~~both every~~ possible values of d_B , and A completes at $2 \leq 2$, always on time, giving expected value $10 + 1 = 11$ —nearly double, by visiting the site with the ~~larger~~ expected deadline first, because d_B ’s tight realization ($d_B = 1$) is only avoidable by acting immediately; information the expected value alone discards. This would couple MWHED to exactly the kind of sequential decision and statistical monitoring questions studied elsewhere in the stochastic vehicle-routing literature, rather than to a direct generalization of Theorems 5–7. A realization in a box $[p_i^-, p_i^+] \times [d_i^-, d_i^+]$ is therefore exactly a feasible set of the MWHED instance (p^+, d^-, w) , and every result above applies to that worst-case instance. This is cheap but conservative; in the case study the worst corner of the sampled ranges (slow roads, early arrival, a 20-minute delay) makes even Paradise individually infeasible, so instance-wise robustness would return nothing. Chance-constrained or scenario-averaged objectives are the useful formulations there.

Calibration and sensitivity. Because the inputs are data-driven, reporting how the optimum moves with them, as in Section 5.7, is part of using the model. The case study shows a practical pattern that scenario models would formalize: the exact optimum of the nominal model inherits the assumed slack, so an optimistic nominal speed can spend the slack of a dominant site on a small one (retaining 31.9% of the drawn optimum), whereas a conservative nominal speed or a plan chosen against a sample of perturbed scenarios retains 99.7% (Section 5.7).

7 Conclusion

We have ~~given a complete complexity and algorithmic characterization of~~ studied Minimum Weighted Hazard-Exposure Dispatch, a scheduling problem that arises ~~naturally~~ whenever a single depot must choose a dispatch order against a spreading hazard with ~~known~~, site-specific arrival deadlines. The problem is exactly the classical weighted on-time-jobs ~~scheduling problem~~ $\text{problem 1} \parallel \sum w_j U_j$ in transportation language, ~~and is weakly NP-hard already in its simplest heterogeneous case (equal deadlines, Theorem 3); an exact~~; ~~consequently its weak NP-hardness, the~~ pseudo-polynomial dynamic program (Theorem 5) and an FPTAS (Theorem 7) give planners exact and provably near-exact tools regardless; and a second, structurally distinct special case—equal dispatch cost, Theorem 8—remains exactly solvable in near-linear time, delineating precisely which combination of network heterogeneity causes the difficulty. These guarantees are not merely convenient: the natural do-nothing fallback

~~of dispatching sites in deadline order with no reconsideration has unbounded~~, the approximation scheme and the matroid greedy algorithm for equal-size sites (for any number of vehicles, identical or not) are classical, and we have presented them with complete, machine-checked proofs rather than as new techniques. The classification of the homogeneous restrictions is an elementary corollary, and not the natural tractability boundary of the scheduling literature, where agreeable weights are the classical polynomial case.

What the dispatch setting adds is modest. Under the arrival reading of the deadline, hardness persists when the hazard-arrival time is the same at every site; the depot round-trip model is a conservative bound on a chained route, which says when its premise is costly (clustered sites); and the FPTAS has the guarantee $\rho_{v,\epsilon} > 1/(1+\epsilon)$, best possible for the scheme as stated and sharper than the textbook $1-\epsilon$, which is not attained. Natural heuristics (deadline order with or without skipping, a weighted greedy repair) have unbounded worst-case ~~loss relative to the optimum~~ (Proposition 9), so some form of optimization is necessary for any performance guarantee at all, and our numerical illustration confirms this qualitative gap persists on random instances of realistic size, alongside a weighted Moore-Hodgson-style greedy repair heuristic that ~~while carrying no proven guarantee~~ recovers substantially ~~more ratio~~, although the greedy repair is within 1% of the optimum ~~than doing nothing at all~~ on random instances; a guarantee requires a rule that reconsiders earlier commitments in light of both weights and dispatch times, and the dynamic program and the FPTAS are two such rules, not the only ones.

~~Section 6 examines three natural relaxations—multiple depots or vehicles, a relocating depot, and uncertain~~ The numerical study and the case study carry two lessons beyond the theorems. The reading of the deadline (the crew must arrive before the hazard, or return before it) changes the answer on the same data and must be stated. And the optimum of a model with estimated inputs inherits the assumed slack: on the Camp Fire instance, built from the hazard-arrival times—~~identifying precisely~~ which part of our analysis (the exchange argument, the dynamic program’s state space, or both) each relaxation breaks, rather than leaving them as an unexamined list.

More broadly, MWHEd is a small, self-contained illustration of a pattern that recurs across combinatorial optimization: a transportation-network reparametrization of a decades-old scheduling problem reveals both that the classical hardness and algorithmic machinery transfer over essentially unchanged (Theorems 3–7), and that the specific structural question of which restriction restores tractability (Theorem 8) has a clean, network-interpretable answer—sites equidistant from the depot—that was not visible in the original scheduling formulation’s language of abstract jobs and machines. We see this as the paper’s main methodological contribution alongside its specific results: reframing a classical scheduling question in transportation terms did not just relabel it, it surfaced a distinction (dispatch-cost heterogeneity versus deadline heterogeneity) that determines tractability, and that distinction is exactly what Section 6’s three extensions each perturb in a different way ~~times that NIST documents for each community, an optimistic nominal speed yields a plan that places a small community ahead of the dominant one with little slack and is fully on time in only 16% of the perturbed scenarios, whereas planning for~~

a conservative speed (which a sample of scenarios also finds) retains almost all of the optimum, in this model. The case study is a single event with four sites, evaluated in a model that is conservative about routing (Proposition 1); it illustrates the model and does not support operational recommendations. Section 6 discusses how scenario-based and robust formulations would capture the brittleness, and which parts of the analysis survive: Lemma 2 holds within a scenario but not across scenarios, which is what makes the stochastic version a genuinely different problem. Natural next steps are a stochastic or chance-constrained MWHED with a decomposition that uses the earliest-deadline lemma within scenarios, a routing variant that chains sites with deadline guarantees, release dates for several vehicles, and approximation schemes for a fixed number of vehicles.

Appendix A Exhaustive verification of the running example

Examples 1–5.4 rely on Example 1’s instance being small enough to check by exhaustive search, not only by the dynamic program itself. Since Lemma 2 lets us restrict to dispatching each candidate subset S in increasing-deadline order, Table A1 lists all $2^4 = 16$ subsets of Example 1’s four sites ($p = (2, 3, 1, 4)$, $d = (4, 6, 2, 9)$, $w = (5, 8, 3, 10)$), each dispatched in that order, together with its total dispatch time and feasibility. Every feasible subset’s weight is at most 23, and $\{1, 2, 4\}$ attains it, confirming $W^* = 23$ independently of Algorithm 1.

Only the full set $\{1, 2, 3, 4\}$ is infeasible, and it is infeasible for exactly the reason Example 1 describes: dispatching all four sites, even in the best possible (deadline) order, makes site 4 complete one time unit after its deadline. Every other subset is feasible, so the optimization reduces to choosing, among the 15 feasible subsets, the one of largest weight—which Table A1 confirms is $\{1, 2, 4\}$, at $W = 23$, exactly matching Example 1, Example 3’s dynamic-program table, and Example 4’s FPTAS instantiation.

Appendix B Elementary derivation of the classical FPTAS bound

Theorem 7’s proof compresses the approximation-ratio derivation into a single chain of inequalities. We spell out each step separately here, for states and proves the sharper guarantee $\rho_{v,\epsilon}$. For a reader who wants to check the bound without re-deriving it—

only the textbook bound $W(\hat{S}) \geq (1 - \epsilon)W^*$ we spell out its short derivation step by step. Recall the setup: $w_{\max} = \max_i w_i$, $K = \max(1, \epsilon w_{\max}/n)$, $w'_i = \lfloor w_i/K \rfloor$, $w_{\max} = \max_i w_i$, $K = \max(1, \epsilon w_{\max}/n)$, $w'_i = \lfloor w_i/K \rfloor$, S^* is the true an optimal subset with $W^* = \sum_{i \in S^*} w_i$, $W^* = \sum_{i \in S^*} w_i$, and $\hat{S}$ is the subset the FPTAS returns (the a feasible subset of largest scaled value v with $g(n, v) < \infty$). As in the main proof, we split on which argument of $K = \max(1, \epsilon w_{\max}/n)$ attains the maximum; the case below covers the case $\epsilon w_{\max}/n < 1$ (where the scaling is vacuous and $\hat{S} = S^*$ exactly) in one line, and spells out the remaining case in full.

~~Vacuous-scaling case~~ Vacuous scaling, $\epsilon w_{\max}/n < 1$ $\epsilon w_{\max} \leq n$. Then $K = 1$, so $w'_i = \lfloor w_i \rfloor = w_i$ (the w_i are already integers) $w'_i = w_i$, the value-indexed dynamic program is identical to Theorem 5's program, and it returns $\hat{S} = S^*$, so $W(\hat{S}) = W^* \geq (1 - \epsilon)W^*$ immediately, since $\epsilon \in (0, 1)$ program is exact, and $W(\hat{S}) = W^*$.

The remaining steps assume $\epsilon w_{\max}/n \geq 1$ From now on $\epsilon w_{\max} \geq n$, so that $K = \epsilon w_{\max}/n$ $K = \epsilon w_{\max}/n$ exactly (not merely as an upper bound) — this exact equality, rather than this equality, not an inequality, is what Step 5 below needs, and is the reason uses, and it is why the case split above is necessary rather than is necessary and not cosmetic.

Step 1: relating w_i and w'_i . By the definition of the floor function, $w'_i \leq w_i/K < w'_i + 1$, so

$$\underline{Kw'_i \leq w_i < K(w'_i + 1)}.$$

$w'_i \leq w_i/K < w'_i + 1$, so

$$\underline{Kw'_i \leq w_i < K(w'_i + 1)}. \quad (\text{B1})$$

Step 2: S^* 's scaled value is close to W^*/K . The right-hand inequality of (B1) rearranged, gives $w_i \leq K(w'_i + 1)$, used in Step 4 below; the left-hand inequality gives $w'_i \leq w_i/K$, used in Step 2.

~~Step 2: S^* 's scaled value is close to W^*/K .~~ Summing the left-hand inequality of (B1)'s consequence $w'_i \leq w_i/K$ is the wrong direction for a lower bound on $\sum_{i \in S^*} w'_i$, so instead we use $w_i/K - 1 < w'_i$ (from $w_i/K < w'_i + 1$, i.e. the same right-hand inequality of (B1) divided by K and rearranged), summed gives $w'_i > w_i/K - 1$. Summing over S^* :

$$\underline{\sum_{i \in S^*} w'_i > \sum_{i \in S^*} \left(\frac{w_i}{K} - 1 \right) = \frac{W^*}{K} - |S^*| \geq \frac{W^*}{K} - n.}$$

$\sim$

$$\underline{\sum_{i \in S^*} w'_i > \sum_{i \in S^*} \left(\frac{w_i}{K} - 1 \right) = \frac{W^*}{K} - |S^*| \geq \frac{W^*}{K} - n.} \quad (\text{B2})$$

Step 3: ~~the FPTAS's returned value $\hat{S}$~~ is at least as good as S^* 's in scaled value. S^* is itself a feasible subset (it is feasible by definition, being the true optimum) feasible, so it attains some scaled value $\sum_{i \in S^*} w'_i =: v_0$ with $g(n, v_0) < \infty$. Since v_0 with $g(n, v_0) < \infty$; $\hat{S}$ is defined as the feasible subset of *largest* has the largest scaled value among all v with $g(n, v) < \infty$, its scaled value is at least v_0 :

$$\underline{\sum_{i \in \hat{S}} w'_i \geq \sum_{i \in S^*} w'_i \stackrel{(\text{B2})}{>} \frac{W^*}{K} - n.}$$

feasible sets, so

$$\sum_{i \in \hat{S}} w'_i \geq \sum_{i \in S^*} w'_i > \frac{W^*}{K} - n. \quad (\text{B3})$$

Step 4: converting $\hat{S}$'s scaled value back to true weight. Using the right-hand inequality of (B1) ($w_i \leq K(w'_i + 1)$) would overcount by Kn if applied termwise and summed; instead we directly lower-bound $\hat{S}$'s true weight using the left-hand inequality of (B1) ($Kw'_i \leq w_i$), summed over $\hat{S}$; and combined with (B3):

$$W(\hat{S}) = \sum_{i \in \hat{S}} w_i \geq K \sum_{i \in \hat{S}} w'_i \stackrel{(\text{B3})}{>} K \left(\frac{W^*}{K} - n \right) = W^* - Kn.$$

gives

$$W(\hat{S}) \geq K \sum_{i \in \hat{S}} w'_i > K \left(\frac{W^*}{K} - n \right) = W^* - Kn. \quad (\text{B4})$$

Step 5: Kn equals $\epsilon w_{\max}$ exactly. In this case $Kn = \epsilon w_{\max}$ exactly (established above), so

$$Kn = \epsilon w_{\max}.$$

This is an equality, not merely a bound—the reason the case split matters: in the vacuum-scaling case K can instead exceed $\epsilon w_{\max}/n$ (since $K=1$ there), which is exactly why that case is handled separately rather than folded into this one $Kn = \epsilon w_{\max}$ exactly, so $Kn = \epsilon w_{\max}$.

Step 6: bounding $w_{\max}$ by W^* . By Assumption 1, the single highest-weight site is individually feasible ($p_i \leq d_i$), so dispatching, so serving it alone is a feasible subset set of value $w_{\max}$; since, and W^* is the optimum over all feasible subsets, $w_{\max} \leq W^*$ at least that.

Combining Steps 4–6:

$$W(\hat{S}) > W^* - Kn = W^* - \epsilon w_{\max} \geq W^* - \epsilon W^* = (1 - \epsilon)W^*,$$

$$W(\hat{S}) > W^* - Kn = W^* - \epsilon w_{\max} \geq W^* - \epsilon W^* = (1 - \epsilon)W^*.$$

which gives $W(\hat{S}) > (1 - \epsilon)W^*$ in this case, and $W(\hat{S}) = W^*$ in the vacuum-scaling case; either way, $W(\hat{S}) \geq (1 - \epsilon)W^*$, matching The proof of Theorem 7 improves this by observing that $\hat{S}$ also has scaled value at least $\lfloor w_{\max}/K \rfloor$, which bounds $W(\hat{S})$ from below by about $w_{\max} - K$ independently of W^* .

Statements and Declarations

Funding

The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

Author Contributions

Q.V.D. conceived the problem and developed the theoretical results (the hardness ~~reduction~~reductions, the exact dynamic program, the FPTAS and its analysis, the matroid-greedy special case, and the ~~naive-policy lower bound~~worst-case examples). H.V.V. implemented the algorithms, ran the numerical experiments and the real-world case study, and produced the figures. M.N.D. conducted the literature review and helped position the work relative to prior scheduling and disaster-response research. P.S.N. designed the real-world case study and its practical interpretation. Q.V.D. wrote the first draft of the manuscript, and all authors commented on previous versions, reviewed, and approved the final manuscript.

Data Availability

~~The synthetic instance generators and all code used to produce every table. The synthetic instances used in Section 5 are included with this submission and are self-contained, except for generated by seeded pseudo-random generators described in that section, and every table and figure is deterministic given those seeds (timings depend on the machine). The real-world instance of Section 5.7 's real-world instance, whose inputs are public-domain figures (is built from published figures: clock times from a NIST technical report and (Maranghides et al. 2021) and 2010 U.S. Census data populations, both cited in the text), rather than data collected by the authors; the script that derives the instance from those public figures and reproduces Tables 10 and 11 is also included with this submission. All source code (the algorithms, the generators, the exhaustive-search verification, the checks of the added results, and the scripts that regenerate every table and figure, including the Camp Fire derivation and sensitivity analysis) and the Lean 4 formalisation are publicly available at https://github.com/vinhqdang/stochastic_vrp, in the directory `papers/csonet2026` (the version corresponding to this manuscript is the latest one at the time of the revision). The statements and proofs of Proposition 1, Lemmas 2 and 6, Theorems 3–8, Propositions 4–11, the arithmetic of the worked examples, and the hardness part of Theorem 12, are machine-checked in the Lean 4 proof assistant with the Mathlib library, with no unproved steps. The formalisation proves the mathematical content (for instance, that the reduction preserves yes/no answers, that the dynamic program computes the optimum, and that the approximation bound holds) and does not cover complexity-class statements (NP-hardness as a class membership, running times) the~~

data-structure implementation of the greedy algorithm, the fact that only the n earliest slots matter, that the dynamic program on the scaled weights of Example 4 selects the stated set, or the modelling remarks; the file `MwheProofs/README.md` lists, theorem by theorem, what is and is not formalised.

Use of Large Language Models

An ~~A~~ generative AI coding assistant (~~Claude, Anthropic~~) was used to assist with LaTeX formatting ~~and~~, to run the numerical ~~illustration~~ code reported in Section 5. ~~All theorem~~, to search and check bibliographic records, and to write the Lean 4 proofs, which the Lean kernel then checks. The authors reviewed all statements, proofs, code and text, and ~~the manuscript text were authored, verified, and are taken responsibility for by the author~~ take responsibility for them.

References

- Avci, M.G., Avci, M., Battarra, M., Erdoğan, G.: The wildfire suppression problem with multiple types of resources. *European Journal of Operational Research* **316**(2), 488–502 (2024) <https://doi.org/10.1016/j.ejor.2024.03.005>
- Alvelos, F., Marto, M., Mendes, A.: Dispatching, positioning and routing resources for wildfire initial attack. *Networks* **86**(1), 80–102 (2025) <https://doi.org/10.1002/net.22278>
- Antonov, N., Šůcha, P., Janota, M., Hůla, J.: Minimizing the weighted number of tardy jobs: data-driven heuristic for single-machine scheduling. *Computers & Operations Research* **185**, 107281 (2026) <https://doi.org/10.1016/j.cor.2025.107281>
- Baptiste, P.: Polynomial time algorithms for minimizing the weighted number of late jobs on a single machine with equal processing times. *Journal of Scheduling* **2**(6), 245–252 (1999) [https://doi.org/10.1002/\(SICI\)1099-1425\(199911/12\)2:6<245::AID-JOS28>3.0.CO;2-5](https://doi.org/10.1002/(SICI)1099-1425(199911/12)2:6<245::AID-JOS28>3.0.CO;2-5)
- Bansal, N., Blum, A., Chawla, S., Meyerson, A.: Approximation algorithms for deadline-TSP and vehicle routing with time-windows. In: *Proceedings of the 36th Annual ACM Symposium on Theory of Computing (STOC '04)*, pp. 166–174 (2004). <https://doi.org/10.1145/1007352.1007385>
- Baptiste, P., Brucker, P., Knust, S., Timkovsky, V.G.: Ten notes on equal-processing-time scheduling: at the frontiers of solvability in polynomial time. *4OR* **2**(2), 111–127 (2004) <https://doi.org/10.1007/s10288-003-0024-4>
- Bringmann, K., Fischer, N., Hermelin, D., Shabtay, D., Wellnitz, P.: Faster minimization of tardy processing time on a single machine. *Algorithmica* **84**(5), 1341–1356 (2022) <https://doi.org/10.1007/s00453-022-00928-w>

- Brucker, P., Kravchenko, S.A.: Scheduling equal processing time jobs to minimize the weighted number of late jobs. *Journal of Mathematical Modelling and Algorithms* **5**(2), 143–165 (2006) <https://doi.org/10.1007/s10852-005-9011-4>
- Chekuri, C., Khanna, S.: A polynomial time approximation scheme for the multiple knapsack problem. *SIAM Journal on Computing* **35**(3), 713–728 (2005) <https://doi.org/10.1137/S0097539700382820>
- Chen, L., Lian, J., Mao, Y., Zhang, G.: A nearly quadratic-time FPTAS for knapsack. *SIAM Journal on Computing* (2025) <https://doi.org/10.1137/24M1686772> . STOC 2024 special issue, article STOC24-1–STOC24-23
- Dessouky, M.I., Lageweg, B.J., Lenstra, J.K., Velde, S.L.: Scheduling identical jobs on uniform parallel machines. *Statistica Neerlandica* **44**(3), 115–123 (1990) <https://doi.org/10.1111/j.1467-9574.1990.tb01276.x>
- Delazeri, G., Ritt, M.: Wildfire Suppression: Complexity, Models, and Instances. *arXiv:2603.29865* (2026)
- Edmonds, J.: Matroids and the greedy algorithm. *Mathematical Programming* **1**(1), 127–136 (1971) <https://doi.org/10.1007/BF01584082>
- Feillet, D., Dejax, P., Gendreau, M.: Traveling salesman problems with profits. *Transportation Science* **39**(2), 188–205 (2005) <https://doi.org/10.1287/trsc.1030.0079>
- Fischer, N., Wennmann, L.: Minimizing tardy processing time on a single machine in near-linear time. *TheoretCS* **4** (2025) <https://doi.org/10.46298/theoretics.25.14>
- Garey, M.R., Johnson, D.S.: *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco (1979)
- Gens, G.V., Levner, E.V.: Fast approximation algorithm for job sequencing with deadlines. *Discrete Applied Mathematics* **3**(4), 313–318 (1981) [https://doi.org/10.1016/0166-218X\(81\)90008-1](https://doi.org/10.1016/0166-218X(81)90008-1)
- Gabow, H.N., Tarjan, R.E.: A linear-time algorithm for a special case of disjoint set union. *Journal of Computer and System Sciences* **30**(2), 209–221 (1985) [https://doi.org/10.1016/0022-0000\(85\)90014-5](https://doi.org/10.1016/0022-0000(85)90014-5)
- Günzel, M., Brüggem, G., Chen, K.-H., Chen, J.-J.: EDF-like scheduling for self-suspending real-time tasks. In: 2022 IEEE Real-Time Systems Symposium (RTSS), pp. 172–184 (2022). <https://doi.org/10.1109/RTSS55097.2022.00024>
- Granda, B., Vitoriano, B., Figueira, J.R.: A mathematical programming approach for a wildfire suppression problem. *Operational Research* **25**(1), 16 (2025) <https://doi.org/10.1007/s12351-024-00882-1>

- Harris, M.G., Forbes, M.A., Taimre, T.: Logic-based Benders decomposition for wild-fire suppression. *Computers & Operations Research* **160**, 106392 (2023) <https://doi.org/10.1016/j.cor.2023.106392>
- Heeger, K., Hermelin, D.: Minimizing the weighted number of tardy jobs is W[1]-hard. In: 32nd Annual European Symposium on Algorithms (ESA 2024). LIPIcs, vol. 308. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2024). <https://doi.org/10.4230/LIPIcs.ESA.2024.68> . Article 68
- Heeger, K., Molter, H.: Minimizing the number of tardy jobs with uniform processing times on parallel machines. In: 42nd International Symposium on Theoretical Aspects of Computer Science (STACS 2025). LIPIcs, vol. 327. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2025). <https://doi.org/10.4230/LIPIcs.STACS.2025.47> . Article 47
- Hermelin, D., Molter, H., Shabtay, D.: Minimizing the weighted number of tardy jobs via $(\max, +)$ -convolutions. *INFORMS Journal on Computing* **36**(3), 836–848 (2024) <https://doi.org/10.1287/ijoc.2022.0307>
- Hejl, L., Šůcha, P., Novák, A., Hanzálek, Z.: Minimizing the weighted number of tardy jobs on a single machine: Strongly correlated instances. *European Journal of Operational Research* **298**(2), 413–424 (2022) <https://doi.org/10.1016/j.ejor.2021.07.002>
- Ibarra, O.H., Kim, C.E.: Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM* **22**(4), 463–468 (1975) <https://doi.org/10.1145/321906.321909>
- Jin, C.: An improved FPTAS for 0-1 knapsack. In: 46th International Colloquium on Automata, Languages, and Programming (ICALP 2019). LIPIcs, vol. 132. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2019). <https://doi.org/10.4230/LIPIcs.ICALP.2019.76> . Article 76
- Karp, R.M.: Reducibility among combinatorial problems. In: Miller, R.E., Thatcher, J.W. (eds.) *Complexity of Computer Computations*, pp. 85–103. Plenum Press, New York (1972). https://doi.org/10.1007/978-1-4684-2001-2_9
- Koca, E.: Two-stage stochastic facility location problem with disruptions and restricted shortages. *Computers & Industrial Engineering* **183**, 109484 (2023) <https://doi.org/10.1016/j.cie.2023.109484>
- Kellerer, H., Pferschy, U.: A new fully polynomial time approximation scheme for the knapsack problem. *Journal of Combinatorial Optimization* **3**(1), 59–71 (1999) <https://doi.org/10.1023/A:1009813105532>
- Kellerer, H., Pferschy, U.: Improved dynamic programming in connection with an FPTAS for the knapsack problem. *Journal of Combinatorial Optimization* **8**(1),

- 5–11 (2004) <https://doi.org/10.1023/B:JOCO.0000021934.29833.6B>
- Klein, K.-M., Polak, A., Rohwedder, L.: On minimizing tardy processing time, max-min skewed convolution, and triangular structured ILPs. In: Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 2947–2960 (2023). <https://doi.org/10.1137/1.9781611977554.ch112>
- Korte, B., Vygen, J.: Combinatorial Optimization: Theory and Algorithms, 6th edn. Algorithms and Combinatorics, vol. 21. Springer, Berlin, Heidelberg (2018). <https://doi.org/10.1007/978-3-662-56039-6>
- Kravchenko, S.A., Werner, F.: Parallel machine problems with equal processing times: a survey. *Journal of Scheduling* **14**(5), 435–444 (2011) <https://doi.org/10.1007/s10951-011-0231-3>
- Lawler, E.L.: Sequencing to minimize the weighted number of tardy jobs. *RAIRO Recherche Opérationnelle* **10**(5, Suppl.), 27–33 (1976)
- Lawler, E.L.: Fast approximation algorithms for knapsack problems. *Mathematics of Operations Research* **4**(4), 339–356 (1979) <https://doi.org/10.1287/moor.4.4.339>
- Liu, C.L., Layland, J.W.: Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM* **20**(1), 46–61 (1973) <https://doi.org/10.1145/321738.321743>
- Lawler, E.L., Moore, J.M.: A functional equation and its application to resource allocation and sequencing problems. *Management Science* **16**(1), 77–84 (1969) <https://doi.org/10.1287/mnsc.16.1.77>
- Lenstra, J.K., Rinnooy Kan, A.H.G., Brucker, P.: Complexity of machine scheduling problems. In: Studies in Integer Programming. *Annals of Discrete Mathematics*, vol. 1, pp. 343–362 (1977). [https://doi.org/10.1016/S0167-5060\(08\)70743-X](https://doi.org/10.1016/S0167-5060(08)70743-X)
- Maranghides, A., Link, E., Mell, W.R., Hawks, S., Wilson, M., Brewer, W., Brown, C., Vihnanek, B., Walton, W.D.: A case study of the camp fire – fire progression timeline. NIST Technical Note 2135, National Institute of Standards and Technology (2021). <https://doi.org/10.6028/NIST.TN.2135>
- Moore, J.M.: An n job, one machine sequencing algorithm for minimizing the number of late jobs. *Management Science* **15**(1), 102–109 (1968) <https://doi.org/10.1287/mnsc.15.1.102>
- Ma, Q., Peng, C., Xu, C., Xu, C., Zhang, R.: Multiagent matroid upgrading: Greedy is fair and efficient. In: Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026), Extended Abstracts. IFAAMAS, ??? (2026)

- Rostamian, P., Ahmadi Dighehsara, A., Yildirim, K.S., Ardestani-Jaafari, A.: Wild-fire management: A systematic review of optimization under uncertainty and complexity. *Operations Research Forum* **7**(1), 18 (2026) <https://doi.org/10.1007/s43069-026-00604-7>
- Rachaniotis, N.P., Pappis, C.P.: Scheduling fire-fighting tasks using the concept of “deteriorating jobs”. *Canadian Journal of Forest Research* **36**(3), 652–658 (2006) <https://doi.org/10.1139/x05-267>
- Sahni, S.K.: Algorithms for scheduling independent tasks. *Journal of the ACM* **23**(1), 116–127 (1976) <https://doi.org/10.1145/321921.321934>
- Shabtay, D., Gaspar, N., Kaspi, M.: A survey on offline scheduling with rejection. *Journal of Scheduling* **16**(1), 3–28 (2013) <https://doi.org/10.1007/s10951-012-0303-z>
- Tezcan, B., Eren, T.: Forest fire management and fire suppression strategies: a systematic literature review. *Natural Hazards* **121**(9), 10485–10515 (2025) <https://doi.org/10.1007/s11069-025-07227-x>
- Terao, T.: Faster matroid partition algorithms. *ACM Transactions on Algorithms* **21**(2), 1–26 (2025) <https://doi.org/10.1145/3707208>
- Tsitsiklis, J.N.: Special cases of traveling salesman and repairman problems with time windows. *Networks* **22**(3), 263–282 (1992) <https://doi.org/10.1002/net.3230220305>
- Vansteenwegen, P., Souffriau, W., Van Oudheusden, D.: The orienteering problem: A survey. *European Journal of Operational Research* **209**(1), 1–10 (2011) <https://doi.org/10.1016/j.ejor.2010.03.045>
- Wang, L.: A two-stage stochastic programming framework for evacuation planning in disaster responses. *Computers & Industrial Engineering* **145**, 106458 (2020) <https://doi.org/10.1016/j.cie.2020.106458>
- Wang, Y., Lv, B., Zhou, Q., Li, J., Tan, T.: Schedulability analysis for self-suspending tasks under EDF-like scheduling. *IEEE Transactions on Computers* **74**(7), 2364–2375 (2025) <https://doi.org/10.1109/TC.2025.3558079>
- Woeginger, G.J.: When does a dynamic programming formulation guarantee the existence of a fully polynomial time approximation scheme (FPTAS)? *INFORMS Journal on Computing* **12**(1), 57–74 (2000) <https://doi.org/10.1287/ijoc.12.1.57.11901>

Table 2 Summary of this section's results. n is the number of sites, $P = \sum_i p_i$, and $\epsilon \in (0, 1)$ is the FPTAS's accuracy parameter.

Setting	Status	Result	Runtime	Origin
General instance	weakly NP-hard	Thm. 3	—	<u>classical</u>
General instance	solved exactly	Thm. 5	$O(nP)$	<u>Lawler–Moore</u>
General instance	$(1-\epsilon)$-approximated <u>$\rho_{n,\epsilon}$-approx.</u> <u>$\rho_{n,\epsilon} > \frac{1}{1+\epsilon}$</u>	Thm. 7	$O(n^3/\epsilon)$	<u>classical scheme, sharper analysis</u>
Equal deadlines $(d_i=D)$	weakly NP-hard (= knapsack)	Thm. 3	—	<u>classical</u>
Equal cost $(p_i=p)$ <u>weights</u>	<u>solved exactly</u>	<u>Thm. 12(a)</u>	<u>$O(n \log n)$</u>	<u>Moore</u>
<u>Equal-size sites, any number of vehicles (identical or not)</u>	solved exactly	Thm. 8	$O(n \log n)$ <u>$O(m + n \log n)$</u>	<u>classical (Lawler; Dessouky et al.)</u>
<u>Which of p, w, d are constant</u>	<u>polynomial iff p or w constant (if $P \neq NP$)</u>	<u>Thm. 12</u>	<u>—</u>	<u>elementary corollary</u>
<u>Constant hazard-arrival time (arrival reading)</u>	<u>weakly NP-hard</u>	<u>Prop. 4</u>	<u>—</u>	<u>new (elementary)</u>
<u>Round-trip vs. chained route</u>	<u>round trip is conservative</u>	<u>Prop. 1</u>	<u>—</u>	<u>new (elementary)</u>
Naive EDD, no repair EDD with skipping	unbounded worst-case ratio	Prop. 9	$O(n \log n)$	<u>illustrative</u>
<u>Weighted greedy repair</u>	<u>unbounded ratio</u>	<u>Prop. 10</u>	<u>—</u>	<u>illustrative</u>
<u>FPTAS guarantee</u>	<u>best possible (without completion step)</u>	<u>Prop. 11</u>	<u>—</u>	<u>new</u>

Table 3 The dynamic program’s table $f(i, t)$ for Example 1 ($i = 1$: site 3, $p = 1, d = 2, w = 3$; $i = 2$: site 1, $p = 2, d = 4, w = 5$; $i = 3$: site 2, $p = 3, d = 6, w = 8$; $i = 4$: site 4, $p = 4, d = 9, w = 10$). A dash marks $-\infty$ (infeasible or unreachable at that cell).

t	0	1	2	3	4	5	6	7	8	9	10
$f(0, t)$	0	–	–	–	–	–	–	–	–	–	–
$f(1, t)$	0	3	–	–	–	–	–	–	–	–	–
$f(2, t)$	0	3	5	8	–	–	–	–	–	–	–
$f(3, t)$	0	3	5	8	11	13	16	–	–	–	–
$f(4, t)$	0	3	5	8	11	13	16	18	21	23	–

Table 4 Mean ratio to the true optimum, by instance size n , over 50 random instances per column (weights ≤ 100). The last three rows show the worst ratio over the 50 instances for greedy repair and for naive EDD, and the share of instances on which the FPTAS’s scaling factor is rounded at all), for $\epsilon = 0.2$ and $\epsilon = 0.1$; here n is the number of sites that remain after individually infeasible ones are deleted, so that the instance is slightly larger.

n	10	15 -20	25 -30-50	75 -100	<u>200</u>	
FPTAS ($\epsilon = 0.2$)	1.000	1.000	1.000	1.000	1.000	1
FPTAS ($\epsilon = 0.1$)	1.000	1.000	1.000	1.000	1.000	1
Greedy repair (M-H style)	0.993	0.997 -0.996	0.993 - 0.997 -0.998- <u>0.999</u>	0.999	0.999	
<u>EDD with skipping</u>	<u>0.933</u>	<u>0.941</u>	<u>0.954</u>	<u>0.972</u>	<u>0.981</u>	
Naive EDD (no repair)	0.380 - <u>0.714</u>	0.343 - <u>0.653</u>	0.622 - <u>0.595</u>	0.369 - <u>0.573</u>	0.323 - <u>0.669</u>	
<u>worst ratio, greedy repair</u>	0.515 - <u>0.929</u>	0.439 - <u>0.963</u>	<u>0.976</u>	<u>0.992</u>	<u>0.995</u>	
<u>worst ratio, naive EDD</u>	<u>0.065</u>	<u>0.132</u>	<u>0.032</u>	<u>0.110</u>	<u>0.027</u>	
<u>share with $K = 1, \epsilon = 0.2 / 0.1$</u>	<u>0/.54</u>	<u>.60/1</u>	<u>1/1</u>	<u>1/1</u>	<u>1/1</u>	

Table 5 FPTAS accuracy with active scaling ($n = 100, p_i \leq 20$ or as stated). Entries are the *worst* ratio to the optimum over the instances of each family (30 per ϵ for the first two). “Share < 1 ” is the fraction of random instances on which the FPTAS was not exactly optimal. Every entry respects the guarantee $1 - \epsilon$.

ϵ	<u>$1 - \epsilon$</u>	<u>Random</u>	<u>Share < 1</u>	<u>Str. corr.</u>	<u>Adv. (analytic)</u>	<u>Adv. + completion</u>
<u>0.50</u>	<u>0.50</u>	<u>0.9998</u>	<u>0.30</u>	<u>0.9993</u>	<u>0.669</u>	<u>1.000</u>
<u>0.30</u>	<u>0.70</u>	<u>0.99996</u>	<u>0.10</u>	<u>0.9994</u>	<u>0.771</u>	<u>1.000</u>
<u>0.20</u>	<u>0.80</u>	<u>0.99996</u>	<u>0.10</u>	<u>0.9994</u>	<u>0.835</u>	<u>1.000</u>
<u>0.10</u>	<u>0.90</u>	<u>0.99999</u>	<u>0.03</u>	<u>0.9996</u>	<u>0.910</u>	<u>1.000</u>
<u>0.05</u>	<u>0.95</u>	<u>1.00000</u>	<u>0.00</u>	<u>0.9996</u>	<u>0.953</u>	<u>1.000</u>

Table 6 Cost of the FPTAS as ϵ shrinks (random weights up to 10^6 , $n = 100$): mean scaling factor K , mean number of dynamic-program cells, and mean wall-clock time. A value-indexed exact table would need about 5×10^7 columns per row at these weights, against the 10^4 – 10^5 used here.

ϵ	0.50	0.30	0.20	0.10	0.05
mean K	4,996	2,997	1,998	999	500
cells ($\times 10^6$)	0.99	1.65	2.47	4.95	9.90
time (ms)	3.8	5.9	8.1	18.3	47.8

Table 7 Mean wall-clock time (milliseconds), $n = 40$, $\epsilon = 0.1$, weights up to 10^6 , over 5 random instances per row, as $p_{\max}$ grows (so P grows roughly proportionally).

$p_{\max}$	50	200	800	3,200	12,800
Exact DP (Theorem 5)	4.7 0.54	16.8 1.26	65.3 3.57	275.7 14.1	1137.9 82.8
FPTAS (Theorem 7)	8.3 1.39	8.4 1.64	7.8 1.86	7.7 1.88	7.6 1.88

Table 8 Mean wall-clock time (milliseconds) of the exact dynamic program and of HiGHS on the subset MIP, 10 random instances per size; the last row gives the slowest MIP solve. Optima agree on all 40 instances.

ϵ 0.50 0.30 n	0.20 20	0.10 50	0.05 100	0.01 200		
Mean ratio to optimum	0.9999 1.0000	Exact DP	1.0000 0.29	1.0000 0.78	1.0000 1.47	1.0000 3.49
Mean time (ms)	HiGHS (MIP)	2.5 55.3	3.9 13.8	3.9 22.7	3.8 146.1	
slowest MIP solve		4.2 514	3.8 49	48	434	

Table 9 Mean ratio to the true optimum under two alternative instance-generation regimes, $n = 30$, 20 random instances per row. “Repair” is the greedy repair of Algorithm 4; “Skip” is EDD with skipping; the FPTAS uses $\epsilon = 0.1$.

Regime	FPTAS ($\epsilon=0.1$)	Repair (M-H)	Skip	Naive EDD
Heavy-tailed weights	1.000	0.998 0.999	0.274 0.944	0.537
Tight deadlines ($0.5 \times$ horizon)	1.000	0.988	0.026 0.799	0.125

Table 10 The Camp Fire instance. d^{haz} is the documented event time in minutes after time zero (06:31); p_i is the round-trip occupancy from the Oroville depot at the two response speeds; the MWHED deadline under the arrival reading is $d_i = d^{\text{haz}} + p_i/2$ (rounded, halves to the nearest even integer); w_i is the 2010 census population.

Site	d^{haz}	p_i (50km/h)	p_i (80km/h)	d_i (50)	d_i (80)	w_i
Concow	54	63	39	52-86	74	710
Paradise	73	70	44	108	95	26,218
Magalia	129	89	55	58-174	156	11,310
Yankee Hill	1,569	54	34	64-333-Paradise-70-44-71-1,596	261,218-586	333

Table 11 On-time weight (population) of each algorithm on the Camp Fire instance, arrival reading; the sites served are in parentheses. The total weight is 38,571. C, P, M, Y denote Concow, Paradise, Magalia, Yankee Hill.

	50 km/h	80 km/h
Exact optimum (Thm. 5)	26,218 (Paradise) 37,861 (P, M, Y)	26,218 (Paradise) 38,571 (all)
FPTAS, $\epsilon = 0.1$ (Thm. 7)	26,218-37,528 (P, M)	26,218-38,238 (C, P, M)
Greedy repair (M-H style)	26,218-37,861	26,218-38,571
Naive EDD (no repair) EDD with skipping	12,353 (C, M, Y)	0-38,571
Naive EDD	710 (Concow)-1,043 (C, Y)	38,571

Table 12 Depot-spoke optimum (the MWHED model) versus the optimum of a chained route, by response speed and road factor ρ

road factor ρ	50	
1.0	37, Magalia 861	38, or Yankee Hill first; any detour through the smaller communities consumes time Paradise
1.3	26,551	
1.6	26,551	

Table A1 All 16 subsets of Example 1's running instance, dispatched in increasing-deadline order. "Sum p " is the subset's total dispatch time; a subset is feasible iff every member completes on time in this order (Lemma 2).

Subset S	Order	Sum p	Feasible?	$W(S)$
$\emptyset$	—	0	yes	0
$\{1\}$	1	2	yes	5
$\{2\}$	2	3	yes	8
$\{3\}$	3	1	yes	3
$\{4\}$	4	4	yes	10
$\{1, 2\}$	1,2	5	yes	13
$\{1, 3\}$	3,1	3	yes	8
$\{1, 4\}$	1,4	6	yes	15
$\{2, 3\}$	3,2	4	yes	11
$\{2, 4\}$	2,4	7	yes	18
$\{3, 4\}$	3,4	5	yes	13
$\{1, 2, 3\}$	3,1,2	6	yes	16
$\{1, 2, 4\}$	1,2,4	9	yes	23
$\{1, 3, 4\}$	3,1,4	7	yes	18
$\{2, 3, 4\}$	3,2,4	8	yes	21
$\{1, 2, 3, 4\}$	3,1,2,4	10	no (site 4 completes at 10 > $d_4=9$)	